

NCP-ArchPreview Technical Report: Moving towards Latent Space Language Models through Next Concept Prediction

The Intern-NCP Team

Shanghai AI Lab; LUMIA Lab, Shanghai Jiao Tong University

 <https://huggingface.co/collections/ArchSpace-Collection/ncp-archpreview>

 <https://github.com/InternLM/lmdeploy>

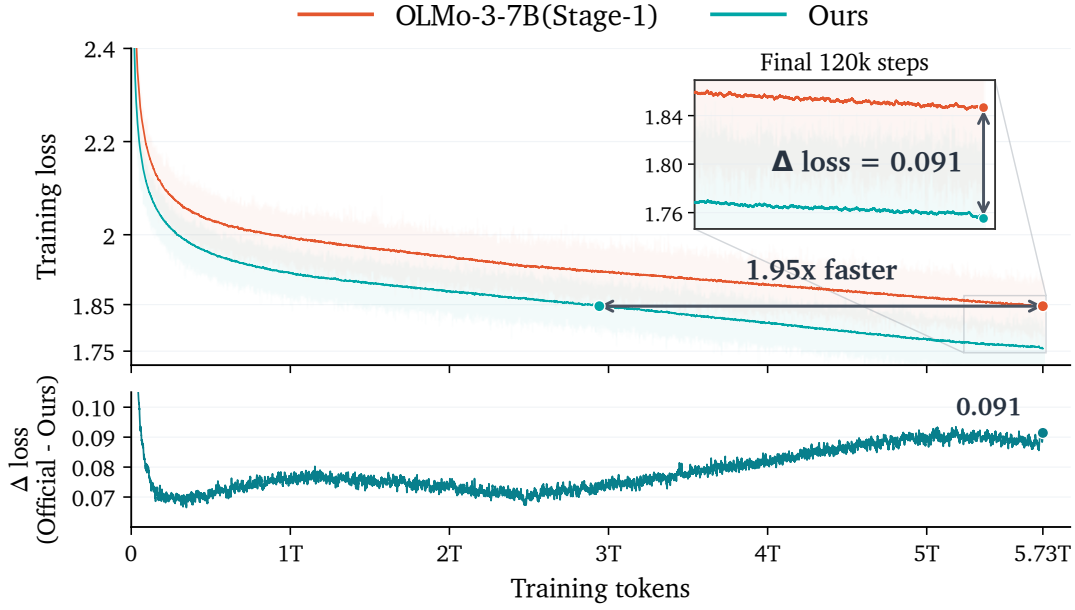
 https://github.com/LUMIA-Group/ncp_olmo_eval

Abstract

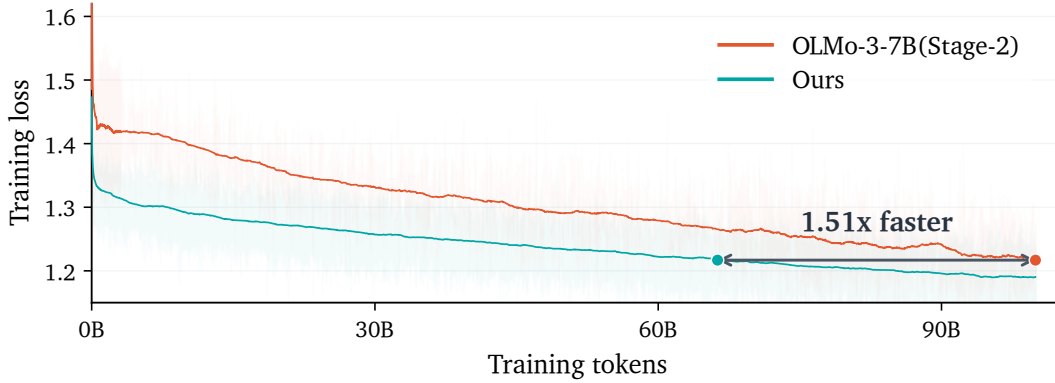
We introduce **NCP-ArchPreview**, a latent-space language model that pushes autoregressive pretraining beyond standard next-token prediction (NTP). Alongside NTP, the model learns through Next Concept Prediction (NCP) to predict discrete concepts that span multiple tokens, introducing an explicit and more challenging concept-level objective while preserving standard token-level autoregressive generation. NCP-ArchPreview builds a latent space by constructing a product-quantized concept vocabulary directly from its hidden states, and subsequently learns to predict future concepts via a dedicated Concept Module. These predicted concepts are then fed back to the token level to guide subsequent generation, with NTP and NCP trained jointly end-to-end. We scale this architecture to 8.9B parameters and train it on 5.73T tokens from the Dolma-3 dataset, marking the largest demonstration of a latent-space language model to date. Remarkably, by consuming only 51.3% of the total training tokens, NCP-ArchPreview achieves the final pretraining loss of OLMo-3-7B. Following full pretraining, it outperforms OLMo-3-7B by 2.45 points on the downstream macro-average, including a notable 5.99-point gain on GSM8K. Controlled experiments isolate a clear progression of performance gains stemming from both the latent architecture and the NCP objective. Furthermore, utilizing only 85% of the standard computation, NCP-ArchPreview approaches the training loss of a strictly parameter-aligned 8.9B baseline. The learned latent space remains highly valuable after the pretraining stage: updating just the 17M-parameter VQ module yields a novel, lightweight interface for domain adaptation, while a simple injection of concept representations into a DFlash2 drafter improves the mean accepted length by 4.17% with negligible overhead. By proving the viability of joint token and concept modeling at scale, these results position latent-space prediction not merely as an auxiliary objective, but a highly efficient and scalable architectural blueprint for next-generation foundation models. We release the Stage-1 checkpoints obtained every 100,000 training steps, the corresponding drafter models, and the final Stage-1 / Stage-2 checkpoint.

1. Introduction

Recent progress in generative AI has demonstrated that the representation space in which a model learns can matter as much as its parameter scale. In visual synthesis, latent diffusion shifts generation from raw pixels into compact continuous representations, dramatically boosting modeling efficiency and scalability [Blattmann et al., 2023, Rombach et al., 2022]. Modern language models similarly induce high-level abstractions, such as semantic concepts and latent world representations, in their



(a) Stage-1 loss curves for OLMo-3-7B and NCP-ArchPreview across all 5.73T pretraining tokens. Despite trained on exactly the same data, NCP-ArchPreview converges 1.95 $\times$ faster and achieves a 0.091 lower loss over the final steps.



(b) Stage-2 loss curves for OLMo-3-7B and NCP-ArchPreview on the same data, showing that NCP-ArchPreview converges 1.51 $\times$ faster with less fluctuations while maintaining a lower training loss.

Figure 1 | Comparison of training dynamics between OLMo-3-7B and our NCP-ArchPreview during pretraining Stage-1 and Stage-2.

hidden states [Gurnee and Tegmark, 2024, Li et al., 2023, Park et al., 2024]. Under standard Next Token Prediction (NTP), however, these abstractions arise purely as an indirect byproduct: supervision is strictly confined to granular tokens, lacking explicit objectives that guide how semantic structure unfolds across multi-token spans. To bridge this gap, we introduce **NCP-ArchPreview**, a novel latent-space foundation model architecture that incorporates the direct prediction of discrete concepts spanning over multiple tokens into the pretraining stage. By jointly pretraining with both NTP and Next Concept Prediction (NCP), NCP-ArchPreview scales latent-space language modeling up to 5.73T pretraining tokens, establishing the architectural viability and scalability of this paradigm at frontier scale.

Prior architectural efforts can be distinguished along two dimensions: how latent representations

are formed, and what they are trained to represent. In terms of latent representation, the Hourglass Transformer [Nawrot et al., 2022], ContextLM [Dai et al., 2025], and MegaByte [Yu et al., 2023] use fixed hierarchical structures, with predefined multiscale resolutions or byte patches. In contrast, the Byte Latent Transformer (BLT) [Pagnoni et al., 2025], DLCM [Qu et al., 2025], and H-Net [Hwang et al., 2026] use dynamic or input-adaptive chunking, allowing latent boundaries or patch sizes to vary with the input. A separate set of methods introduces new latent prediction targets or training objectives. Large Concept Model [Barrault et al., 2024] predicts representations in a shared continuous sentence-embedding space; and ConceptLM [Liu et al., 2026] predicts discrete concepts from a learned concept vocabulary.

Joint-Embedding Predictive Architectures (JEPA) [LeCun et al., 2022] offer a principled alternative: predict future representations directly in latent space to capture invariant, semantic structures. This paradigm has driven breakthroughs in image representation learning (I-JEPA) [Assran et al., 2023] and large-scale video modeling (V-JEPA and V-JEPA 2) [Assran et al., 2025, Bardes et al., 2024]. In language modeling, however, learning explicitly upon latent targets remains underexplored. While Multi-Token Prediction (MTP) [Gloeckle et al., 2024] supervises multiple future positions, its loss is still tethered to individual surface tokens. NCP-ArchPreview extends this principle to language modeling via Next Concept Prediction (NCP) [Liu et al., 2026], which predicts latent representations directly over a learned concept vocabulary.

Built upon the OLMo-3-7B backbone [Olmo et al., 2026], NCP-ArchPreview establishes an 8.9B-parameter realization of this architecture. It inserts an 8-layer Concept Module between a 16-layer Token Encoder and a 16-layer Token Decoder, adopts product quantization to construct an expressive discrete concept space, and incorporates multiway dynamic dense connections adapted from MUDDFormer [Xiao et al., 2025] to facilitate hierarchical residual routing. Extensive pretraining across 5.73T tokens reveals substantial optimization efficiency: NCP-ArchPreview matches the final pretraining loss of OLMo-3-7B using only 51.3% of its training tokens (1.95× convergence speedup), and outperforms OLMo-3-7B by 2.45 points on downstream macro-averages (+5.99 on GSM8K) after pretraining. Compute-aligned and parameter-aligned ablations confirm that these gains stem fundamentally from the latent hierarchy and the NCP objective rather than superficial capacity increases, yielding a 1.74× improvement in Pareto compute efficiency. Beyond pretraining, the learned concept space serves as an efficient 17M-parameter interface for lightweight domain adaptation and accelerates inference by +4.17% mean accepted length when integrated into a DFlash2 speculative drafter [Inco AI, 2026].

To accelerate open-source research on non-vanilla foundation architectures, with this paper we formally release **NCP-ArchPreview** as a fully pretrained latent-space foundation model, including its model weights, inference scripts, training recipes, and intermediate evaluation checkpoints. In the remainder of this report, we provide a comprehensive technical account of this release: Section 2 details the modular architecture and hierarchical residual routing; Section 3 introduces the joint NTP-NCP training objective; and Section 4 presents empirical validations across trillion-token pretraining, component ablations, scaling laws, and downstream evaluations. We further investigate practical training considerations, including attention-logit stabilization under matrix optimizers and capability-aware proxy metrics for mid-training data curation, before discussing downstream adaptation mechanisms, multi-token prediction synergies, and future directions for latent-space language modeling.

2. Model Architecture

We introduce NCP-ArchPreview, a latent-space language model that couples token-level modeling with concept-level prediction in a discrete latent space. We construct a discrete concept vocabulary from intermediate token representations, use a Concept Module to predict future concepts, and feed those predictions back into the token stream. Hierarchical residual further mixes information across depths and modules. The following subsections define these components.

2.1. Overview

Our model introduces a latent-space prediction to a standard NTP model. The backbone is organized into three modules: a Token Encoder, a Concept Module, and a Token Decoder. The Token Encoder produces token representations, and then the model compresses contiguous groups of tokens into continuous concept representations. The Concept Module predicts the next concept from the preceding concepts, while the Token Decoder uses the predicted concepts together with token-level representations to predict the next token. In addition, our hierarchical residual connects layers within each module and across the three modules, allowing information to travel across depth and hierarchy.

Let k be the concept compression factor. Let $M = \lfloor T/k \rfloor$. The Token Encoder maps the input sequence to token-level hidden states $\mathbf{h}_{1:T}$. Then we use mean pooling over each group of k states to produce a continuous concept sequence $\mathbf{c}_{1:M}$. We use Vector Quantization (VQ) to learn a finite concept space, forming the concept vocabulary [Gray, 1984]. The Concept Module then predicts a differentiable representation of the next concept by a weighted combination of codebook entries. Before being injected into the Token Decoder, the predicted concept sequence is repeated at token resolution, shifted to preserve causality. The corresponding training objectives are described in Sec. 3.

2.2. Learning the Discrete Concept Vocabulary with Vector Quantization

The VQ objective encourages the codebook entries to cover the distribution of continuous concept representations, thereby providing a structured target space for prediction. Then, the Concept Module predicts the next concept in this learned space. It produces a probability distribution over each codebook, whose weighted combination gives a differentiable concept prediction. This subsection describes the codebook learning.

The Token Encoder is the first module of the backbone and produces token-level hidden states

$$\mathbf{h}_{1:T} = \text{TokenEncoder}_{\theta_e}(x_{1:T}), \quad \mathbf{h}_t \in \mathbb{R}^d. \quad (1)$$

These hidden states serve three purposes: they are consumed by the Token Decoder, compressed into concept-level representations for the Concept Module, and used to learn the concept codebooks.

For the m -th concept, we aggregate a contiguous group of k token states using mean pooling:

$$\mathbf{c}_m = f_c(\mathbf{h}_{(m-1)k+1:m}) = \frac{1}{k} \sum_{i=1}^k \mathbf{h}_{(m-1)k+i}, \quad \mathbf{c}_m \in \mathbb{R}^d. \quad (2)$$

The resulting concept sequence has length $M = \lfloor T/k \rfloor$, which is shorter than the token sequence by a factor of k .

We use VQ to map each continuous concept representation to a finite set of representative vectors. To increase the capacity of this discrete concept space without using a large monolithic codebook, we

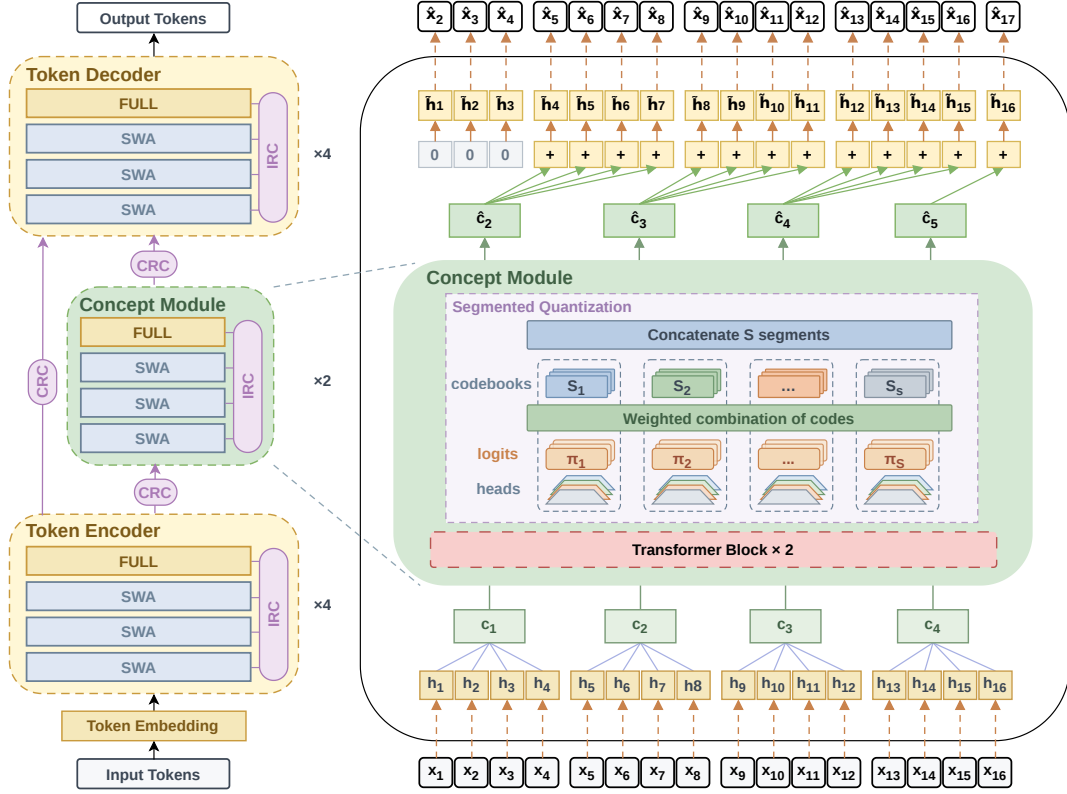


Figure 2 | Overview of NCP-ArchPreview. The token-level module (comprising the Token Encoder and Token Decoder) maintains token-level representations, ensuring the overall input and output remain consistent with canonical autoregressive decoding. In the intermediate layers, the Concept Module compresses token spans into concepts and predicts the next concept via weighted combinations of entries from a vector-quantized concept vocabulary, which is learned from the Token Encoder’s hidden states. Hierarchical residual connections, including Cross-Module Residual Connections (CRC) and Intra-Module Residual Connections (IRC), allow information to flow smoothly across layers and abstraction levels.

adopt product quantization [Jegou et al., 2010]. Each concept vector is partitioned into S feature segments:

$$\mathbf{c}_m = \text{concat}(\mathbf{c}_m^1, \dots, \mathbf{c}_m^S), \quad \mathbf{c}_m^s \in \mathbb{R}^{d/S}. \quad (3)$$

For segment s , let $\mathcal{E}^s = \{\mathbf{e}_1^s, \dots, \mathbf{e}_N^s\}$ denote its codebook. We assign each continuous segment to its nearest codeword:

$$n_m^s = \arg \min_{n \in \{1, \dots, N\}} \|\mathbf{c}_m^s - \mathbf{e}_n^s\|_2^2, \quad (4)$$

$$\mathbf{d}_m^s = \mathbf{e}_{n_m^s}^s. \quad (5)$$

The complete quantized concept is obtained by concatenating the quantized segments:

$$\mathbf{d}_m = \text{concat}(\mathbf{d}_m^1, \dots, \mathbf{d}_m^S). \quad (6)$$

Although each segment codebook contains only N entries, product quantization defines N^S possible combinations of segment codewords. It therefore substantially increases the capacity of the discrete concept space while keeping the codebooks small. The quantization loss and its stop-gradient operations are given in Sec. 3.

2.3. Predicting the Next Concept with the Learned Vocabulary

Given the learned codebooks from Sec. 2.2, the Concept Module predicts the next concept autoregressively in the resulting structured latent space. During training, it consumes the continuous concept history $\mathbf{c}_{<m}$ and is supervised toward the next concept. At inference time, previously predicted concepts are fed back autoregressively. Given the history $\mathbf{c}_{<m}$, a stack of Transformer layers produces a latent hidden state

$$\mathbf{u}_m = \text{ConceptModule}_{\theta_c}(\mathbf{c}_{<m}). \quad (7)$$

For every product quantization segment s , a segment-specific prediction head produces a distribution over the codebook entries $\mathcal{E}^s$:

$$\boldsymbol{\pi}_m^s = \text{softmax}(\text{PredictionHead}_c^s(\mathbf{u}_m)), \quad \boldsymbol{\pi}_m^s \in \mathbb{R}^N. \quad (8)$$

Rather than taking an argmax or sampling a codeword, we form a differentiable predicted segment by taking the expectation under this distribution:

$$\hat{\mathbf{c}}_m^s = \sum_{n=1}^N \pi_{m,n}^s \mathbf{e}_n^s. \quad (9)$$

The predicted concept is the concatenation of all segment-wise predictions:

$$\hat{\mathbf{c}}_m = \text{concat}(\hat{\mathbf{c}}_m^1, \dots, \hat{\mathbf{c}}_m^S). \quad (10)$$

This weighted-codeword construction keeps the prediction path fully differentiable while restricting it to the linear combination of the learned codebooks. The resulting NCP objective therefore supervises prediction in a constrained latent space instead of regressing to an unconstrained, freely drifting continuous target.

2.4. Injecting Predicted Concepts into the Token Stream

The span of predicted concepts must align with the token-level states consumed by the Token Decoder. We first repeat every predicted concept k times and then apply a causal shift. Let $\Delta = k$ denote the shift used by our implementation, for token-level prediction positions $t = 1, \dots, T - 1$, the token-level concept signal is

$$\mathbf{b}_t = \begin{cases} \mathbf{0}, & 1 \leq t < \Delta, \\ \hat{\mathbf{c}}_{\lfloor (t-\Delta)/k \rfloor + 2}, & \Delta \leq t < T. \end{cases} \quad (11)$$

The shift is introduced to prevent information leakage: each predicted concept is injected only after the token states used to predict its target have been processed.

The concept and token hidden states share the same hidden dimension d ; so we directly fuse the two embeddings, with an element-wise residual addition:

$$\tilde{\mathbf{h}}_t = \mathbf{h}_t + \mathbf{b}_t. \quad (12)$$

The Token Decoder then predicts the next token autoregressively from the fused states,

$$p(x_{t+1} \mid x_{\leq t}) = P_{\theta_d}(\tilde{\mathbf{h}}_{\leq t}). \quad (13)$$

2.5. Hierarchical Residual Connections

The Token Encoder, Concept Module, and Token Decoder operate at different depths and sequence granularities. We therefore introduce hierarchical residual connections to facilitate information flow both within and across modules. Inspired by the dynamic dense connections of MUDDFormer [Xiao et al., 2025], our design uses hidden-state-dependent connection weights while retaining a single hidden stream in each module. The proposed connections have two complementary components: Intra-Module Residual Connections (IRC), which combine representations from different layers within the same module, and Cross-Module Residual Connections (CRC), which transfer representations between modules.

2.5.1. Intra-Module Residual Connections

Let $s \in \{\text{TokenEncoder}, \text{ConceptModule}, \text{TokenDecoder}\}$ denote a module. For layer ℓ , let $\mathbf{H}_\ell^s$ be the input to its Transformer block and let

$$\mathbf{R}_\ell^s = F_\ell^s(\mathbf{H}_\ell^s) \quad (14)$$

denotes the block output before the residual addition. In a standard Transformer, the next hidden state is obtained by adding the block output to the current state:

$$\mathbf{H}_{\ell+1}^s = \mathbf{H}_\ell^s + \mathbf{R}_\ell^s. \quad (15)$$

Our Intra-Module Residual Connections (IRC) generalize this fixed depth-wise addition by allowing the next state to combine representations from multiple depths. At layer ℓ , we construct the candidate set

$$\mathcal{X}_\ell^s = \{\mathbf{H}_1^s, \mathbf{H}_1^s + \mathbf{R}_1^s, \dots, \mathbf{H}_\ell^s + \mathbf{R}_\ell^s\}. \quad (16)$$

A lightweight MLP applied to the current block output $\mathbf{R}_\ell^s$ produces one coefficient for each candidate state:

$$\mathbf{w}_\ell^s = \text{MLP}_\ell^s(\mathbf{R}_\ell^s). \quad (17)$$

The next hidden state is their weighted combination:

$$\mathbf{H}_{\ell+1}^s = \sum_{j=1}^{\ell+1} w_{\ell,j}^s \mathbf{X}_{\ell,j}^s. \quad (18)$$

The coefficients are not normalized, so IRC can represent signed combinations of states from different depths. We initialize the MLP such that $\mathbf{w}_\ell^s = [0, \dots, 0, 1]$. Thus, IRC initially selects the most recent residual-updated state and reduces to the standard residual connection, while training can learn to incorporate information from earlier layers.

2.5.2. Cross-Module Residual Connections

Cross-Module Residual Connections (CRC) transfer information between modules. Let

$$\mathbf{S}^u = \{\mathbf{S}_j^u\}_{j=1}^K$$

denote the representations exported by a source module u , and let $\mathbf{T}_\ell^s$ denote the state of a target module s . When the two modules operate at different sequence granularities, the source representations are first aligned by chunking or repetition. A lightweight MLP applied to the target state then produces normalized coefficients over the source depths:

$$\alpha_\ell^{s \leftarrow u} = \text{softmax}(\text{MLP}_\ell^{s \leftarrow u}(\mathbf{T}_\ell^s)), \quad \mathbf{M}_\ell^{s \leftarrow u} = \sum_{j=1}^K \alpha_{\ell,j}^{s \leftarrow u} \text{LN}(\mathbf{S}_j^u). \quad (19)$$

The resulting representation is added to the target stream using a learned diagonal scaling:

$$\mathbf{T}_{\ell, \text{out}}^s = \mathbf{T}_{\ell}^s + \mathbf{D}_{\ell}^{s \leftarrow u} \odot \mathbf{M}_{\ell}^{s \leftarrow u}, \quad \mathbf{D}_{\ell}^{s \leftarrow u} \in \mathbb{R}^d. \quad (20)$$

We use three cross-module connections:

$$\begin{aligned} \text{TokenEncoder} &\rightarrow \text{ConceptModule} \\ \text{TokenEncoder} &\rightarrow \text{TokenDecoder} \\ \text{ConceptModule} &\rightarrow \text{TokenDecoder} \end{aligned}$$

The ConceptModule-to-TokenDecoder connection follows the same causal shift as the predicted-concept pathway and therefore does not expose future-token information. The diagonal scalings are initialized with small values, so the model starts close to the original backbone and gradually learns to exploit the additional cross-module connections. Together, IRC and CRC define the hierarchical residual connection mechanism.

3. Training

3.1. End-to-End Training Overview

The Concept Module is trained jointly with the token-level model. For an input sequence $x_{1:T}$, the Token Encoder first produces token-level states $\mathbf{h}_{1:T}$, and the pooling operator in Sec. 2.2 produces continuous concept states $\mathbf{c}_{1:M}$. The VQ module assigns each concept to a structured codebook representation, the Concept Module predicts the next concept from the concept history, and the causally shifted predictions are injected into the Token Decoder. The resulting token-level and concept-level predictions are under an end-to-end optimization of the concept pathway. The codebooks, Token Encoder, Concept Module, and Token Decoder are updated together by NTP and NCP objectives.

3.2. Learning the Discrete Concept Vocabulary

Our VQ codebooks provide a structured target space for concept prediction. For each product quantization segment, the codebook is trained to fit the distribution of continuous concept representations:

$$\mathcal{L}_{\text{VQ}} = \frac{1}{MS} \sum_{m=1}^M \sum_{s=1}^S \|\text{sg}(\mathbf{c}_m^s) - \mathbf{d}_m^s\|_2^2, \quad (21)$$

where $\text{sg}(\cdot)$ denotes stop-gradient and $\mathbf{d}_m^s$ is the nearest codebook entry assigned to $\mathbf{c}_m^s$. The stop-gradient operation prevents the VQ objective from updating the Token Encoder. Consequently, this objective only moves the selected codebook entries toward the continuous concept representations, allowing the codebooks to track the latent distribution without directly altering the token-level hidden states.

3.3. Next Concept Prediction

The goal of next-concept prediction is to model dependencies between successive groups of tokens at the concept level. Because each concept summarizes multiple token positions, this objective provides supervision over dependencies spanning multiple tokens. A direct regression objective over continuous concept representations would allow the model to produce arbitrary vectors in the representation space. Such an objective specifies only the distance to the target and does not define which outputs constitute valid concept representations.

We therefore parameterize the prediction through the learned VQ codebooks. For each product quantization segment, the Concept Module predicts a probability distribution over the corresponding codebook entries. The entries are then combined according to the predicted distribution, yielding a differentiable concept representation. This design provides a structured set of reference representations while avoiding a non-differentiable discrete selection step.

Specifically, we train the Concept Module to predict the next continuous concept using a mean-squared error objective:

$$\mathcal{L}_{\text{NCP}} = \frac{1}{M-1} \sum_{m=2}^M \|\hat{\mathbf{c}}_m - \text{sg}(\mathbf{c}_m)\|_2^2. \quad (22)$$

Here, $\mathbf{c}_m$ is the continuous concept computed from the m -th token group, while $\hat{\mathbf{c}}_m$ is predicted from the preceding concept history.

The NCP loss updates both the Concept Module and the Token Encoder. The target $\mathbf{c}_m$ is detached. The Token Encoder receives NCP gradients only through the preceding concept representations $\mathbf{c}_{<m}$ supplied to the Concept Module. This encourages the Token Encoder to retain information that is useful for predicting the next concept.

3.4. Next Token Prediction

The fused representation $\tilde{\mathbf{h}}_t$ from Eq. 12 is trained with the usual causal next-token objective:

$$\mathcal{L}_{\text{NTP}} = -\frac{1}{T-1} \sum_{t=1}^{T-1} \log p_{\theta_d}(x_{t+1} | \tilde{\mathbf{h}}_{\leq t}). \quad (23)$$

The zero prefix and shift ensure that the NTP loss never conditions on a concept computed from the tokens whose prediction it is meant to supervise. NTP provides dense supervision for every weight in the model. It updates the token pathway and the concept pathway while preserving the standard autoregressive language-modeling behavior of the backbone.

3.5. Joint Objective and Gradient Flow

We jointly optimize the model with

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{NTP}} + \alpha \mathcal{L}_{\text{NCP}} + \beta \mathcal{L}_{\text{VQ}}, \quad (24)$$

where $\mathcal{L}_{\text{NTP}}$ trains the model for next-token prediction, $\mathcal{L}_{\text{NCP}}$ trains the Concept Module and Token Encoder for next-concept prediction, and $\mathcal{L}_{\text{VQ}}$ fits the codebook entries to the continuous concept representations. The coefficients α and β control the weights of the two auxiliary objectives. The three objectives are jointly optimized during training.

3.6. Optimization

We use Moonlight Muon [Liu et al., 2025], built on the Muon optimizer [Jordan et al., 2024], for the matrix-valued parameters. For a matrix parameter $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, the update can be written as

$$\mathbf{W}_t = \mathbf{W}_{t-1} - \eta_t \left(\lambda_{\text{Muon}} \mathbf{O}_t \sqrt{\max(d_{\text{in}}, d_{\text{out}})} + \lambda_{\text{wd}} \mathbf{W}_{t-1} \right), \quad (25)$$

where $\mathbf{O}_t$ is the orthogonalized Muon update, η_t is the learning-rate schedule, λ_{Muon} controls the update scale, and λ_{wd} denotes weight decay. Our default setting uses a learning rate of 6×10^{-5} and the same cosine scheduler as Olmo-3-7B. Embeddings, biases, and other non-Muon parameters are optimized with AdamW; other parameters are optimized by Muon.

4. Experiments

4.1. Experimental Setup

Models and Datasets: We use OLMo-3-7B as the backbone of NCP-ArchPreview and follow the staged data curriculum of OLMo-3. The first training stage uses Dolma 3 Mix, and the second stage continues training on Dolma 3 Dolmino. We evaluate the resulting models following the OLMo evaluation protocol implemented in OLMo-Core, which covers 30 benchmark families, including MMLU [Hendrycks et al., 2021a], GSM8K [Cobbe et al., 2021], MATH-500 [Hendrycks et al., 2021b, Lightman et al., 2023], HumanEval [Chen et al., 2021], MBPP [Austin et al., 2021], ARC [Clark et al., 2018], and HellaSwag [Zellers et al., 2019]. These benchmarks evaluate factual knowledge, mathematical reasoning, code generation, commonsense reasoning, reading comprehension, and language modeling. The complete task list, prompting configurations, and evaluation metrics are provided in Appendix E.

For the VQ training study in Section 5.1, we continue training the Stage-1 checkpoint separately on Magicoder [Wei et al., 2023], Orca-Math [Mitra et al., 2024], and TriviaQA-RC [Joshi et al., 2017] for code, mathematics, and knowledge adaptation, respectively. Target-domain performance is evaluated using HumanEval and MBPP with their EvalPlus variants for code [Austin et al., 2021, Chen et al., 2021], GSM8K and MATH-500 for mathematics [Cobbe et al., 2021, Hendrycks et al., 2021b, Lightman et al., 2023], and TriviaQA for knowledge [Joshi et al., 2017].

Configuration for NCP-ArchPreview: NCP-ArchPreview retains the main configuration of OLMo-3-7B, including a hidden size of 4,096, an FFN size of 11,008, and 32 attention heads. Its 32 token-level Transformer layers are divided evenly into a 16-layer Token Encoder and a 16-layer Token Decoder. The Concept Module consists of 8 Transformer layers and uses a chunk size of 4 tokens. The VQ module contains 32 codebooks, each with 128 entries of dimension 128. Concept representations are incorporated through normalized residual connections at every decoder layer. The training objective combines the standard next-token prediction loss with the VQ and NCP losses. The resulting model contains approximately 8.94 billion parameters and uses a maximum context length of 8,192 tokens.

4.2. Main Results

4.2.1. Overview Across Training Stages

We compare NCP-ArchPreview with OLMo-3-7B across the first two stages of training. Figure 1 compares their token-level language-modeling losses, while Table 1 summarizes downstream performance under the same evaluation protocol. Overall, NCP-ArchPreview achieves a lower language-modeling loss and a higher macro-average over the downstream metrics in both stages.

4.2.2. Training Loss Performance

NCP-ArchPreview exhibits lower training loss than OLMo-3-7B throughout Stage-1 and Stage-2, as shown in Figure 1. Both models are trained on the same data. During Stage-1, the loss gap between NCP-ArchPreview and OLMo-3-7B widens as training progresses, reaching 0.091 by the end of the stage. NCP-ArchPreview reaches the final loss of OLMo-3-7B after consuming only 51.3% of the training tokens, corresponding to $1.95\times$ faster convergence in terms of training tokens. The optimization advantage remains evident in Stage-2: NCP-ArchPreview achieves a 0.027 lower final loss while matching the final loss of OLMo-3-7B with only 66.2% of the training tokens, yielding a $1.51\times$ convergence speedup.

Table 1 | Downstream performance grouped by task domain. Vanilla denotes the released OLMo-3 model for each stage. Metrics above the likelihood block are reported as percentages and are higher-is-better. The likelihood block reports bits per UTF-8 byte (BPB), where lower is better. The BPB average is computed separately and is excluded from Overall AVG.

Category	Dataset	Stage-1		Stage-2	
		Vanilla	NCP	Vanilla	NCP
MMLU	MMLU-Humanities	64.73	68.28 +3.55	69.29	70.78 +1.49
	MMLU-Social Sciences	71.18	73.97 +2.79	74.27	76.79 +2.52
	MMLU-STEM	53.04	55.53 +2.49	59.89	61.01 +1.12
	MMLU-Other	64.00	65.63 +1.63	66.40	68.84 +2.44
	MMLU-Pro	19.53	20.26 +0.73	21.77	22.29 +0.52
	MMLU	62.22	64.80 +2.58	66.66	68.48 +1.82
	AVG	54.50	56.73 +2.24	58.32	59.94 +1.62
MATH	GSM8K	39.27	45.26 +5.99	79.68	83.02 +3.34
	GSM-Symbolic	18.85	22.80 +3.95	57.32	60.32 +3.00
	Minerva	12.52	15.63 +3.11	42.09	42.32 +0.23
	MATH-500	12.52	14.48 +1.96	43.44	43.91 +0.47
	AVG	20.79	24.54 +3.75	55.63	57.39 +1.76
Code	BigCodeBench	20.68	22.79 +2.11	39.58	38.42 -1.16
	HumanEval	27.10	31.38 +4.28	49.31	45.62 -3.69
	DS-1000	18.58	20.40 +1.82	25.10	26.00 +0.90
	MBPP	34.53	35.91 +1.38	48.98	50.85 +1.87
	MultiPL-E HumanEval	21.34	22.27 +0.93	31.92	30.90 -1.02
	MultiPL-E MBPP	28.67	33.96 +5.29	41.64	40.85 -0.79
	AVG	25.15	27.79 +2.64	39.42	38.77 -0.65
MC-STEM	ARC-E	90.95	92.30 +1.35	93.81	94.07 +0.26
	ARC-C	77.99	81.57 +3.58	85.49	83.28 -2.21
	AVG	84.47	86.93 +2.47	89.65	88.67 -0.98
MC-Non-STEM	PiQA	72.25	80.85 +8.60	78.35	81.45 +3.10
	CommonsenseQA	72.65	74.04 +1.39	78.05	77.48 -0.57
	SocialIQA	65.35	69.24 +3.89	74.16	74.36 +0.20
	AVG	70.08	74.71 +4.63	76.85	77.76 +0.91
GenQA	HellaSwag	66.05	66.05 +0.00	65.60	66.40 +0.80
	WinoGrande	49.17	51.22 +2.05	50.04	49.64 -0.40
	LAMBADA-Standard	66.60	66.89 +0.29	65.55	67.13 +1.58
	LAMBADA-OpenAI	72.13	72.21 +0.08	69.22	69.96 +0.74
	MedMCQA	32.98	33.21 +0.23	32.36	32.99 +0.63
	MedQA	38.81	38.96 +0.15	38.18	38.10 -0.08
	AVG	54.29	54.76 +0.47	53.49	54.04 +0.55
Overall AVG		46.59	49.04 +2.45	56.98	57.57 +0.59
Likelihood(↓)	HumanEval Gold	0.384	0.364 -0.020	0.362	0.339 -0.023
	MBPP Gold	0.475	0.473 -0.002	0.443	0.454 +0.011
	MT-MBPP Gold	0.437	0.434 -0.004	0.402	0.426 +0.025
	CoQA	0.366	0.370 +0.004	0.305	0.345 +0.040
	DROP	4.474	4.397 -0.077	4.431	4.146 -0.285
	GSM8K Gold	0.405	0.400 -0.005	0.356	0.381 +0.024
	Jeopardy	0.352	0.347 -0.005	0.385	0.358 -0.027
	LAMBADA-OpenAI	0.295	0.288 -0.007	0.320	0.307 -0.014
	Natural Questions	0.917	0.884 -0.033	0.841	0.777 -0.064
	SQuAD	0.140	0.151 +0.011	0.083	0.098 +0.015
	AVG	0.824	0.811 -0.014	0.793	0.763 -0.030

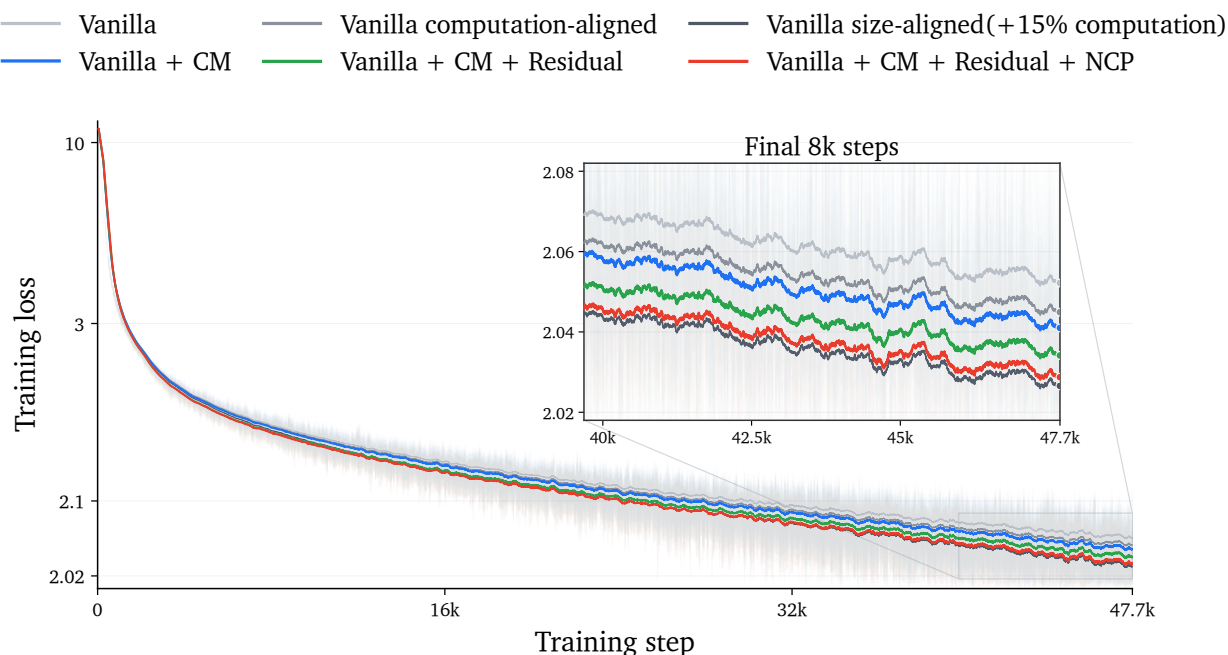


Figure 3 | Training-loss ablation of NCP-ArchPreview over the first 200B training tokens. All runs use the same training recipe. The six curves correspond to three standard OLMo-3-7B baselines Vanilla, Vanilla computation-aligned, and Vanilla size-aligned (+15% computation) and three progressive configurations Vanilla+CM, Vanilla+CM+Residual, and Vanilla+CM+Residual+NCP, with the last corresponding to NCP-ArchPreview. The inset enlarges the final 8k training steps to better resolve the differences among the configurations.

4.2.3. Downstream Performance

NCP-ArchPreview performs better than OLMo-3-7B on downstream tasks at both stages. At Stage-1, NCP-ArchPreview outperforms OLMo-3-7B on almost all evaluated benchmarks, with an average score that is 2.45 points higher across all evaluated datasets. The improvements are particularly large on MATH, Code, and MC-Non-STEM. Among them, the largest improvement is on MATH, reaching 18%. This is consistent with the lower training loss we observe.

At Stage-2, NCP-ArchPreview achieves a 0.59-point higher macro-average, with improvements on MATH, MC-Non-STEM, and GenQA. However, Stage-2 contains only approximately 10% code data. Better fitting of the aggregate mixture can therefore improve the overall average while weakening performance on underrepresented domains such as code.

4.2.4. Relationship Between Training Loss and Downstream Performance

We examine whether training loss at different training stages accurately reflects model capability. At Stage-1, we evaluate checkpoints saved at different points during training on the same downstream tasks (See Appendix E for detailed downstream scores). As training progresses, the loss gradually decreases, and the downstream scores continuously increase, indicating that model capability improves throughout training. At Stage-2, we experiment with three training configurations, V1-V3. Although their final training losses become progressively lower, their downstream performance becomes progressively worse. We believe that this phenomenon may result from a mismatch between the Stage-2 training data distribution and the downstream-task data distribution. We discuss this issue further with a proxy dataset in Appendix C.

Table 2 | Model configurations under parameter- and computation-aligned settings. P_{blk} and F_{blk} denote the parameter count and computation of one standard OLMo-3-7B Transformer block, respectively. The Vanilla size-aligned baseline uses 40 standard OLMo-3-7B blocks to match the approximate parameter count of NCP-ArchPreview, while the Vanilla computation-aligned baseline uses 34 blocks to match its approximate computation.

Model	Token Encoder	Token Decoder	Concept Module	Parameters	Computation
NCP-ArchPreview	$16P_{\text{blk}}$	$16P_{\text{blk}}$	$8P_{\text{blk}}$	$40P_{\text{blk}}$	$34F_{\text{blk}}$
Vanilla	$32P_{\text{blk}}$	–	–	$32P_{\text{blk}}$	$32F_{\text{blk}}$
Vanilla size-aligned	$40P_{\text{blk}}$	–	–	$40P_{\text{blk}}$	$40F_{\text{blk}}$
Vanilla computation-aligned	$34P_{\text{blk}}$	–	–	$34P_{\text{blk}}$	$34F_{\text{blk}}$

4.3. Ablation Studies

We ablate the key components of NCP-ArchPreview and conduct parameter- and computation-aligned comparisons against standard OLMo-3. Specifically, we first compare NCP-ArchPreview with size-aligned and computation-aligned standard OLMo-3-7B models, and then examine the contributions of the Concept Module, residual connections, and the NCP loss through progressive component ablations.

4.3.1. Matched Model Size and Computation

To evaluate whether the improvement of NCP-ArchPreview comes from the proposed architecture rather than simply from additional parameters or computation, we compare it with three OLMo-3-based baselines under the same experimental settings: Vanilla, Vanilla size-aligned, and Vanilla computation-aligned.

As described in the architecture section, the Concept Module groups every four token-level hidden states into one concept representation, reducing the Concept Module sequence length to approximately one quarter of the original sequence. Thus, each Concept Module block introduces roughly the same number of parameters as a standard OLMo-3-7B block while requiring only about one quarter of its computation.

For clarity, we denote the parameter count and computation of one standard OLMo-3-7B Transformer block as P_{blk} and F_{blk} , respectively. Accordingly, one Concept Module block contributes approximately P_{blk} parameters but less than one quarter of its analytical training FLOPs. Since the VQ codebook and residual components account for only a negligible fraction of the total parameters and computation, we omit them when constructing the matched baselines. Detailed model configurations are shown in Table 2.

As shown in Figure 3, NCP-ArchPreview substantially outperforms both Vanilla and Vanilla computation-aligned, indicating that its improvement cannot be explained solely by additional computation. Moreover, NCP-ArchPreview approaches the performance of Vanilla size-aligned, despite using only $34/40 = 85\%$ of its computation.

4.3.2. Module Ablation

We further analyze the contributions of the key components in NCP-ArchPreview by progressively introducing them into the standard OLMo-3-7B architecture. Specifically, we compare Vanilla, Vanilla+CM, Vanilla+CM+Residual, and Vanilla+CM +Residual+NCP, where CM denotes the

Concept Module and the final configuration corresponds to NCP-ArchPreview.

As shown in Figure 3, introducing the Concept Module improves the training loss over Vanilla. Adding the residual component further reduces the loss, and incorporating the NCP loss yields an additional improvement. These results show that the components of NCP-ArchPreview progressively improve model performance, with the complete model approaching the Vanilla size-aligned baseline.

4.4. Hierarchical Residual Connection Comparison

We evaluate variants of the hierarchical residual connections on the 1B-scale NCP-ArchPreview model after 150B training tokens. The reported loss is averaged over the final 200 logged steps. The model without hierarchical residual connections serves as the common reference for both loss and computation. We report the loss difference and additional analytical training FLOPs relative to this reference.

Input-Level Cross-Module Connections augment Intra-Module Residual Connections (IRC) with three embedding-level connections: Token Encoder embedding $\rightarrow$ Concept Module embedding, Concept Module embedding $\rightarrow$ Token Decoder embedding, and Token Encoder embedding $\rightarrow$ Token Decoder embedding. Block AttnRes [Team et al., 2026b] groups residual updates into blocks of four and also includes cross-module connections. Table 3 compares these variants with IRC and the complete hierarchical residual connection design.

IRC + CRC achieves the largest loss reduction, improving over the reference without hierarchical residual connections by 0.0323 with an additional 0.051% analytical training FLOPs. IRC alone achieves a 0.0273 loss reduction with only 0.024% additional FLOPs. Adding the three input-level cross-module connections further improves the loss by 0.0023, achieving a 0.0296 reduction while introducing no additional counted tensor contractions. Applying softmax normalization at all connection stages gives a similar result, with a loss that is only 0.0003 higher at the same analytical FLOPs.

The Block AttnRes variant includes cross-module connections from the Token Encoder to the Concept Module and from the Token Encoder and Concept Module to the Token Decoder. With a comparable 0.026% FLOPs increase, it achieves a 0.0180 loss reduction, which is smaller than the improvement of the IRC-based variants. These analytical FLOPs do not account for source-state materialization, memory traffic, reduction operations, or small-kernel launch overhead. The complete hierarchical connection design may therefore incur additional runtime and memory costs beyond those reflected in the analytical FLOPs.

4.5. Scaling Law

For comparison with OLMo-3 as the compute budget increases, we conduct a series of scaling experiments at multiple FLOPs budgets [Olmo et al., 2026]. For each budget, we search over training hyperparameters and the allocation between model size and training data, then report the best validation loss found at that budget. As shown in Figure 4, NCP-ArchPreview achieves 1.74x computational efficiency compared to OLMo-3 with compute-optimal training. The details are described in Appendix D.

4.6. Model Analysis

Table 3 | Ablation of hierarchical residual connections on the 1B-scale NCP-ArchPreview model after 150B training tokens. Loss is averaged over the final 200 logged steps. Negative loss differences indicate improvements over the reference without hierarchical residual connections; positive FLOPs differences indicate additional analytical training computation.

Variant	Avg. LM loss	Δ loss vs. reference	Δ FLOPs vs. reference
NCP-ArchPreview (without hierarchical residual connections)	2.2588	0.0000	0.000%
IRC + CRC	2.2265	-0.0323	+0.051%
IRC only	2.2315	-0.0273	+0.024%
IRC + Input-Level Cross-Module Connections	2.2292	-0.0296	+0.024%
IRC + All-Stage Softmax	2.2295	-0.0294	+0.024%
Block AttnRes ($S = 4$) + Cross-Module Connections	2.2409	-0.0180	+0.026%

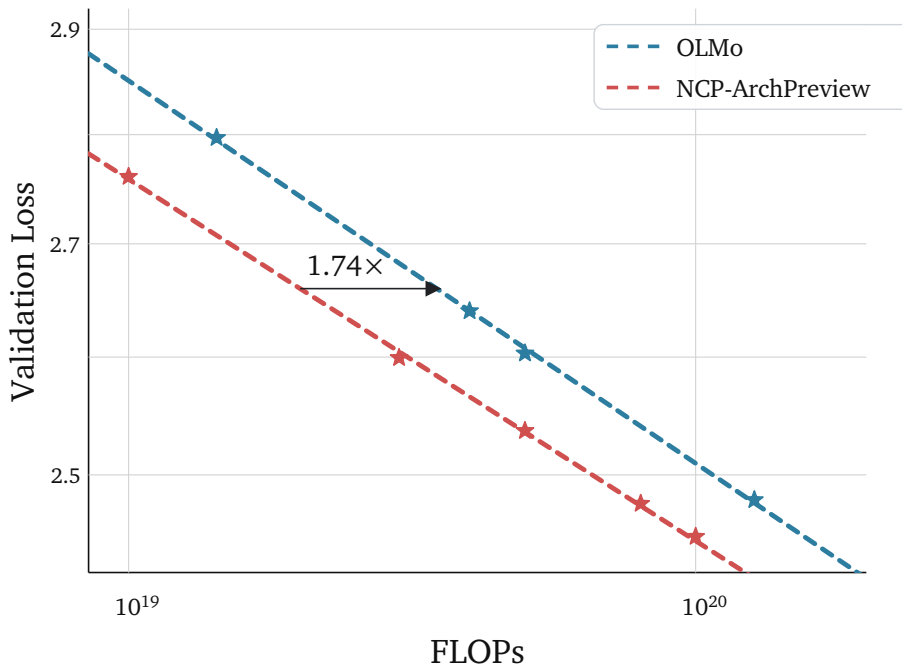


Figure 4 | The scaling law curves for OLMo-3 and NCP-ArchPreview. Each marker denotes the best validation loss found after searching training hyperparameters and model/data allocations at a fixed FLOPs budget. Dashed curves show the corresponding scaling fits.

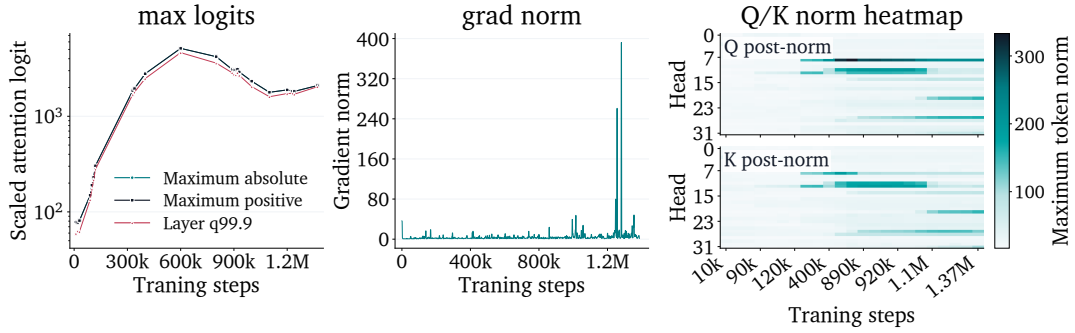


Figure 5 | Numerical instabilities observed when using the OLMo-3-7B layer-wise Q/K normalization. Exploding attention logits (left) coincide with severe gradient-norm spikes (middle) and head-wise outliers in the post-normalization Q/K norms (right).

4.6.1. Numerical Stability of the OLMo-3-7B Configuration

When training NCP-ArchPreview with the layer-wise Q/K normalization adopted by OLMo-3, we observe persistent growth of attention logits. A pronounced imbalance in the matrix norms of the Q/K head blocks accompanies this growth. A small number of outlier heads can dominate the pre-softmax scores, leading to highly concentrated attention distributions. The enlarged Q/K states are also associated with uneven gradient flow, abnormal Q/K gradient norms, and eventual spikes in the global gradient norm.

4.6.2. Ablation of the Instability

To identify the source of this behavior, we compare matched AdamW and Muon baselines and evaluate Muon variants replacing layer-wise Q/K normalization with per-head Q/K normalization. All other training settings remain unchanged. The Muon baseline already reproduces the instability. In contrast, NCP-ArchPreview performs better in terms of stability. Per-head Q/K normalization consistently suppresses both Q/K norm outliers and attention-logit growth.

These results suggest that the primary source of the instability is the interaction between full-matrix Muon updates and layer-wise Q/K normalization. Full-matrix Muon couples the updates of multiple attention heads, whereas layer-wise normalization constrains only their aggregate scale. It does not independently control the scale or QK alignment of each head, allowing a small number of heads to dominate and produce large dot products progressively. This interpretation is consistent with prior observations that Muon can be susceptible to attention-logit explosion concentrated in a subset of heads [Team et al., 2026a].

Per-head Q/K normalization provides a more effective stabilization method in this analysis. Nevertheless, we use the layer-wise Q/K normalization adopted by OLMo-3-7B in all main experiments to preserve a controlled comparison with the original baseline. The per-head variant is reported only as a stabilization intervention in the ablation study, and is not used to obtain the main performance results.

5. Additional Results

5.1. VQ Training

The discrete concept vocabulary provides NCP-ArchPreview with a compact interface between token representations and concept-level prediction. We investigate whether this interface can also

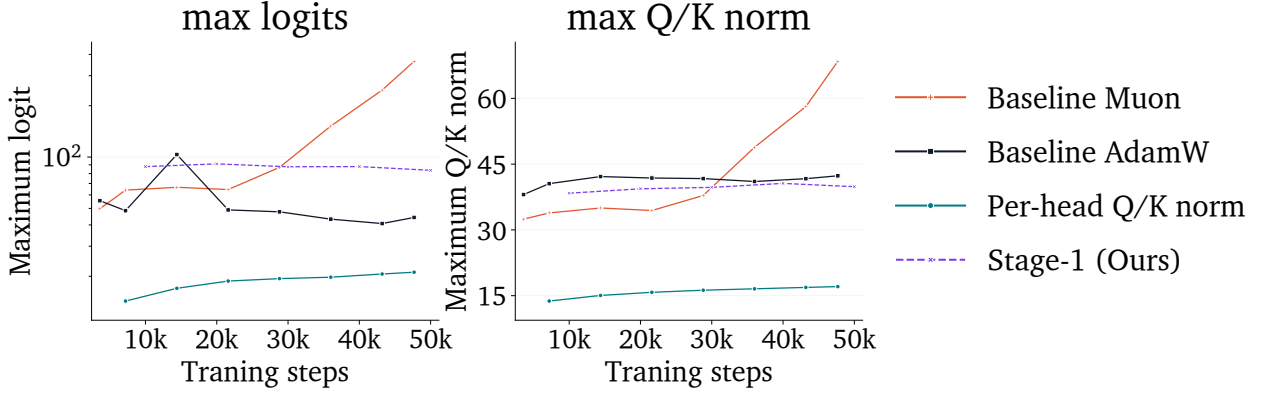


Figure 6 | Ablation of the numerical instability. The maximum absolute attention logit (left) and post-normalization Q/K norm (right) grow under the Muon baseline, whereas per-head Q/K normalization keeps both bounded.

Table 4 | Parameters of the three training strategies. VQ and Full training update existing weights without adding parameters, whereas LoRA introduces more parameters.

Method	Trainable Parameters	Added Parameters
+Full	8.9B	0
+LoRA	17M	17M
+VQ	17M	0

support lightweight domain adaptation. All adaptation experiments start from the corresponding Stage-1 checkpoints of OLMo-3-7B and NCP-ArchPreview.

For NCP-ArchPreview, the +VQ setting freezes the token-level backbone and updates only the VQ codebooks and concept-prediction heads, corresponding to 17M trainable parameters. We compare this setting with full-parameter training (+Full) and low-rank adaptation (+LoRA) [Hu et al., 2021]. To provide a parameter-matched comparison, we configure LoRA with 17M trainable parameters. We also apply Full and LoRA training to OLMo-3-7B using the same domain-specific data and training settings. Table 4 summarizes the parameter costs. The +VQ and +LoRA settings each update 17M parameters, while +Full updates the entire model. VQ adaptation introduces no new parameters because it updates existing codebooks and prediction heads, whereas LoRA adds 17M parameters to the model.

Code: Table 5 reports adaptation on code benchmarks. For NCP-ArchPreview, VQ training raises the code average from 30.04 to 32.69 (+2.65), the highest among the adapted variants, and is the only setting that improves all four code tasks. Full training improves HumanEval, HumanEval+, and MBPP, but its 22.49-point drop on MBPP+ lowers the aggregate below its Stage-1 baseline. LoRA also loses 7.67 points on MBPP+. Against the matched OLMo-3-7B controls, the Full and LoRA variants of NCP-ArchPreview achieve higher code averages and higher general averages. Such post-adaptation regressions are consistent with interference documented in domain adaptation and continual learning, where optimization on new data can reduce previously acquired capabilities [Cheng et al., 2024, Kirkpatrick et al., 2017, Van de Ven et al., 2024]. Although VQ training leaves the token-level backbone unchanged, limiting interference with previously learned code capabilities, its general average nevertheless decreases by 0.42 points.

Table 5 | Code adaptation from Stage-1 across four programming benchmarks, together with performance retained on general benchmarks.

Model	Code					General					
	HumanEval	HumanEval+	MBPP	MBPP+	Avg.	MMLU	ARC-E	HellaSwag	NQ Open	SciQ MC	Avg.
OLMo-3-7B	26.94	23.17	34.27	28.04	28.11	62.25	91.08	66.25	29.46	88.50	67.51
+Full	53.35 +26.41	15.85 -7.32	38.88 +4.61	2.38 -25.66	27.62 -0.49	61.43 -0.82	90.70 -0.38	65.85 -0.40	28.22 -1.24	88.20 -0.30	66.88 -0.63
+LoRA	34.05 +7.11	19.51 -3.66	37.81 +3.54	6.35 -21.69	24.43 -3.68	61.13 -1.12	91.12 +0.04	65.15 -1.10	28.06 -1.40	88.70 +0.20	66.83 -0.68
NCP-ArchPreview	31.27	23.17	36.09	29.63	30.04	64.77	92.89	65.85	32.10	87.80	68.68
+Full	43.27 +12.00	23.78 +0.61	39.53 +3.44	7.14 -22.49	28.43 -1.61	63.24 -1.53	92.76 -0.13	65.55 -0.30	30.15 -1.95	89.30 +1.50	68.20 -0.48
+LoRA	36.49 +5.22	28.05 +4.88	39.02 +2.93	21.96 -7.67	31.38 +1.34	63.76 -1.01	92.26 -0.63	65.35 -0.50	32.17 +0.07	89.30 +1.50	68.57 -0.11
+VQ	33.94 +2.67	28.66 +5.49	36.94 +0.85	31.22 +1.59	32.69 +2.65	63.71 -1.06	92.17 -0.72	65.95 +0.10	31.95 -0.15	87.50 -0.30	68.26 -0.42

Table 6 | Mathematics adaptation from the corresponding Stage-1 checkpoints on GSM8K and MATH-500, with general benchmarks measuring retained capabilities.

Model	Math			General						
	GSM8K	MATH-500	Avg.	MMLU	ARC-E	HellaSwag	NQ Open	SciQ MC	Avg.	
OLMo-3-7B	39.88	12.79	26.34	62.25	91.08	66.25	29.46	88.50	67.51	
+Full	63.23 +23.35	9.84 -2.95	36.54 +10.20	60.93 -1.32	90.74 -0.34	65.20 -1.05	28.29 -1.17	86.90 -1.60	66.41 -1.10	
+LoRA	47.61 +7.73	14.48 +1.69	31.05 +4.71	61.21 -1.04	91.20 +0.12	64.50 -1.75	29.39 -0.07	87.20 -1.30	66.70 -0.81	
NCP-ArchPreview	46.78	14.34	30.56	64.77	92.89	65.85	32.10	87.80	68.68	
+Full	61.03 +14.25	12.40 -1.94	36.72 +6.16	63.90 -0.87	92.34 -0.55	64.90 -0.95	29.91 -2.19	87.50 -0.30	67.71 -0.97	
+LoRA	53.07 +6.29	14.34 +0.00	33.71 +3.15	64.02 -0.75	92.09 -0.80	65.15 -0.70	31.75 -0.35	88.30 +0.50	68.26 -0.42	
+VQ	52.69 +5.91	16.96 +2.62	34.83 +4.27	64.11 -0.66	92.72 -0.17	66.55 +0.70	32.79 +0.69	89.20 +1.40	69.07 +0.39	

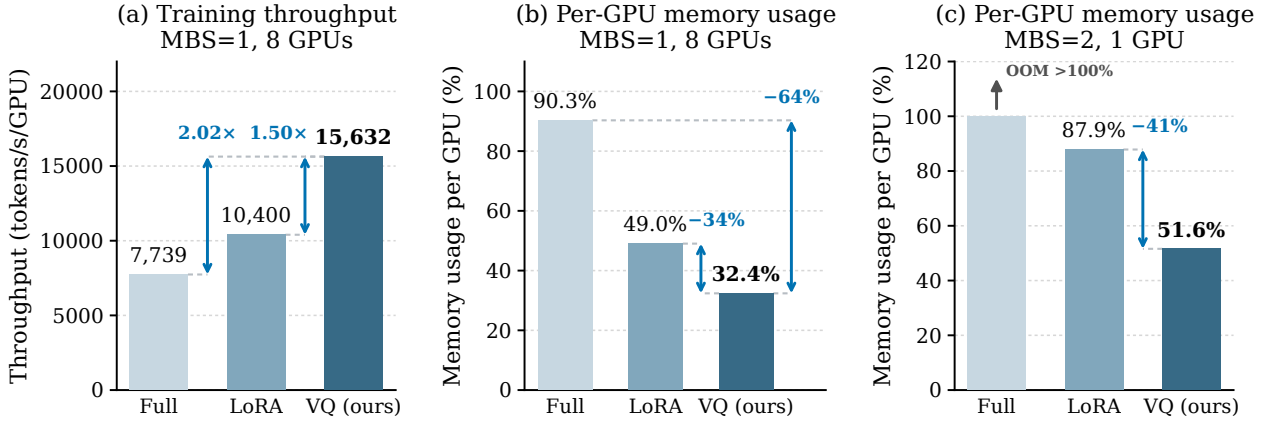
Math: As shown in Table 6, VQ training improves the math average from 30.56 to 34.83 (+4.27). This gain exceeds LoRA (+3.15), although it remains below full training (+6.16). The forgetting behavior reverses this ordering: the general average changes by +0.39 for VQ, compared with -0.42 for LoRA and -0.97 for full training. VQ therefore exceeds LoRA on both reported averages. Relative to full training, it trades 1.89 points of target performance for a 1.36-point higher general average. The matched comparison also favors NCP-ArchPreview: Full and LoRA reach higher math averages than their OLMo-3 counterparts, while retaining higher general averages.

Knowledge: For knowledge adaptation, Table 7 shows that, for NCP-ArchPreview, VQ training improves TriviaQA exact match from 40.28 to 49.47 (+9.19), while keeping the general average essentially unchanged (+0.03). The target gain is smaller than those of full training (+18.00) and LoRA (+17.48). Relative to OLMo-3, NCP-ArchPreview obtains larger TriviaQA gains with both full training (+18.00 vs. +8.53) and LoRA (+17.48 vs. +8.06). OLMo-3-7B also forgets more: its general average drops by 1.04 and 1.80 points, respectively, with especially large SciQ MC losses of 4.90 and 9.60 points. Prior work characterizes Transformer feed-forward layers as key-value memories and identifies specific FFN neurons and mid-layer MLPs that mediate factual recall and support factual editing [Dai et al., 2022, Geva et al., 2021, Meng et al., 2022]. VQ training leaves these backbone MLP parameters fixed and adapts only the concept vocabulary. It therefore cannot directly rewrite factual associations encoded in the backbone, which accounts for its smaller TriviaQA gain relative to full training and LoRA.

Training Throughput and Memory Usage: Figure 7 compares the training throughput and GPU memory usage of VQ, LoRA, and full training. With a micro-batch size of 1 on 8 GPUs, VQ reaches 15,632 tokens/s/GPU, compared with 10,400 for LoRA and 7,739 for full training, corresponding to 1.50× and 2.02× higher throughput, respectively. Although VQ and LoRA both optimize 17M parameters, VQ achieves 50% higher throughput, while full training updates all 8.9B parameters. VQ also has the lowest memory footprint, using 32.4% of the available memory, compared with 49.0% for LoRA and 90.3% for full training. With a micro-batch size of 2 on a single GPU, VQ uses 51.6% of

Table 7 | Results after knowledge adaptation, initialized from the corresponding Stage-1 checkpoints.

Model	Knowledge	General					
	TriviaQA EM	MMLU	ARC-E	HellaSwag	NQ Open	SciQ MC	Avg.
OLMo-3-7B	48.49	62.25	91.08	66.25	29.46	88.50	67.51
+Full	57.02 +8.53	61.73 -0.52	90.57 -0.51	66.65 +0.40	29.80 +0.34	83.60 -4.90	66.47 -1.04
+LoRA	56.55 +8.06	62.19 -0.06	90.82 -0.26	67.50 +1.25	29.13 -0.33	78.90 -9.60	65.71 -1.80
NCP-ArchPreview	40.28	64.77	92.89	65.85	32.10	87.80	68.68
+Full	58.28 +18.00	64.70 -0.07	92.76 -0.13	67.05 +1.20	30.62 -1.48	86.80 -1.00	68.39 -0.29
+LoRA	57.76 +17.48	64.07 -0.70	92.51 -0.38	67.10 +1.25	31.14 -0.96	87.90 +0.10	68.54 -0.14
+VQ	49.47 +9.19	64.37 -0.40	92.38 -0.51	66.65 +0.80	31.63 -0.47	88.50 +0.70	68.71 +0.03


Figure 7 | Training speed and per-GPU memory usage for full training, LoRA, and VQ training. Speed is measured in tokens/s/GPU, memory is reported as a percentage of capacity, and MBS denotes micro-batch size.

memory, while LoRA reaches 87.9% and full training exceeds memory capacity. Overall, VQ combines parameter efficiency relative to full training with higher throughput and lower memory usage than both baselines.

5.2. Multi-Token Prediction on NCP-ArchPreview

We build NCP-ArchPreview and its baseline from OLMo-3-3B. The 3B-scale NCP-ArchPreview organizes the model into an 8-layer Token Encoder, a 4-layer Concept Module, and a 7-layer Token Decoder. For a fair comparison, we further modify OLMo-3-3B by dividing its 16 Transformer layers into groups of 8/1/7 layers and placing the same hierarchical residual connections (Section 2.5) at corresponding depths, which is denoted as OLMo-3-3B + Residual. For each architecture, we train a variant with one additional MTP layer. All four controlled variants are trained on the first 150B tokens of the pretraining corpus. Following DeepSeek-V3 [DeepSeek-AI et al., 2025], the main model retains the next-token prediction (NTP) objective, while the added MTP layer predicts the token two positions ahead with loss weight 0.3.

Figure 8 shows how the ordering develops over training. OLMo-3-3B + Residual with MTP initially has lower loss than NCP-ArchPreview, but NCP-ArchPreview crosses below it near update 2.285k. Over the final 8k updates, the curves consistently maintain the ordering NCP-ArchPreview with MTP, NCP-ArchPreview, OLMo-3-3B + Residual with MTP, and OLMo-3-3B + Residual, from lowest to highest loss. NCP-ArchPreview without MTP eventually outperforms OLMo-3-3B + Residual with

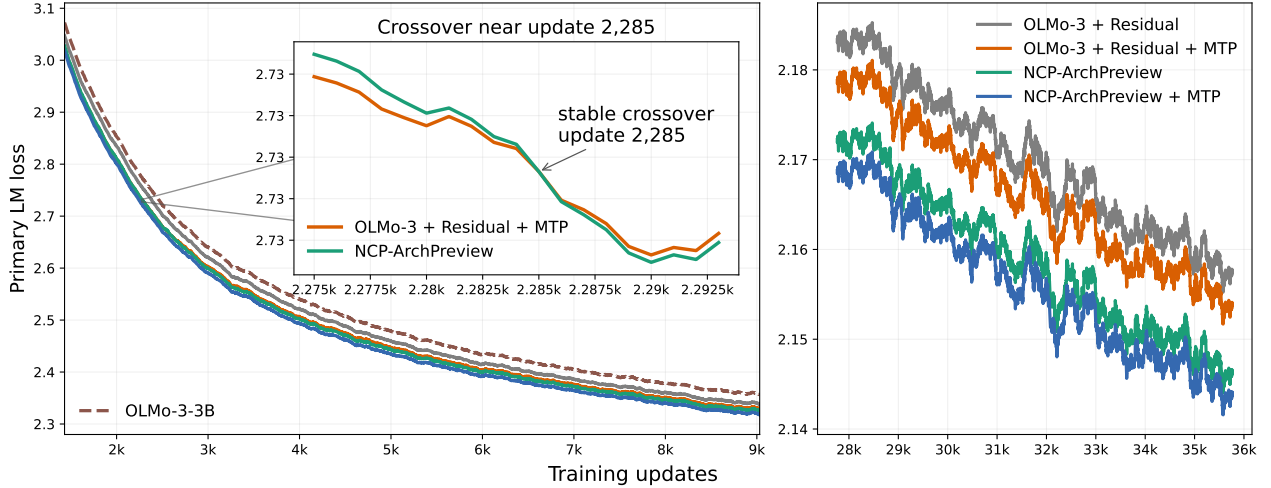


Figure 8 | NTP loss over the first 150B training tokens, smoothed with a 200-update exponential moving average. The left panel includes the crossover in early training. The right panel shows the final 8k updates of the compared variants.

Table 8 | Comparison of residual-aligned OLMo-3-3B variants and the 3B-scale NCP-ArchPreview model. LM loss is averaged over the first 150B training tokens and excludes auxiliary losses.

Model	Avg. LM loss	Δ loss vs. OLMo-3 + Residual	Training FLOPs	Δ FLOPs vs. OLMo-3 + Residual
OLMo-3-3B + Residual	2.3805	0.0000	3.245×10^{21}	0.00%
OLMo-3-3B + Residual + MTP	2.3739	-0.0065	3.750×10^{21}	+15.55%
NCP-ArchPreview	2.3707	-0.0098	3.227×10^{21}	-0.57%
NCP-ArchPreview + MTP	2.3664	-0.0141	3.731×10^{21}	+14.98%

MTP as training progresses, while adding MTP to NCP-ArchPreview provides a further improvement.

Table 8 summarizes the complete training interval over the first 150B tokens. NCP-ArchPreview improves over OLMo-3-3B + Residual by 0.0098, which is greater than the 0.0065 improvement from adding MTP to OLMo-3-3B + Residual. NCP-ArchPreview with MTP is 0.0075 lower than OLMo-3-3B + Residual with MTP while using slightly fewer training FLOPs.

5.3. Predicted Concept Improves Block-Parallel Speculative Drafting

Recent advances in block-diffusion language modeling have demonstrated the feasibility of generating multiple tokens in parallel within each block [Arriola et al., 2025, Cheng et al., 2026a]. Building on this capability, block-parallel draft models have emerged as a promising direction for speculative decoding [Chen et al., 2026, Cheng et al., 2026b, Inco AI, 2026]. These models reduce drafting latency by proposing an entire token block in one forward pass, but their efficiency remains limited by the difficulty of maintaining a coherent prediction trajectory across multiple future positions.

The results in Section 5.2 suggest that concept representations contain information useful for multi-token prediction. We therefore investigate whether the states produced by the Concept Module can improve the acceptance length of a block-parallel Drafter.

Table 9 | Mean accepted length on the evaluation set.

Benchmark	Baseline	Baseline+Concept	Relative gain
GSM8K	6.351	6.537	+2.93%
MATH	6.105	6.240	+2.22%
HumanEval	5.432	5.845	+7.59%
MBPP	5.844	6.099	+4.37%
Macro average	5.933	6.180	+4.17%

Experimental Design: Our baseline combines complementary architectural improvements from DFlash2 and DFlare. Following DFlash2 [Inco AI, 2026], we use a two-tap dynamic convolution to model local dependencies between adjacent draft positions and a lightweight path selector to select a coherent sequence from the top-16 candidates at each position. Following DFlare [Zhang et al., 2026], each Drafter layer learns its own weighted combination of five Target hidden states, providing layer-specific access to Target representations at different semantic depths.

Our *Baseline+Concept* variant keeps the baseline unchanged and adds a causal hierarchical signal. For a verified prefix ending at position t , we use the concept representation $c_{[t/k]}$ from the last fully completed Target chunk. At Drafter layer ℓ , the concept representation is incorporated into every proposal position as

$$\tilde{s}_{t,j}^{(\ell)} = s_{t,j}^{(\ell)} + \tanh(g^{(\ell)}) \odot \text{RMSNorm}_{\ell}(c_{[t/k]}), \quad j = 1, \dots, K. \quad (26)$$

The gates are initialized to zero, allowing the Drafter to gradually incorporate the concept signal. This adds only 0.04M parameters to the 1.1B-parameter Drafter, without introducing additional Target-model computation.

Experimental setting: The *Baseline* and *Baseline+Concept* Drafters are trained using the same online Target distillation pipeline, data order, random seed, and training budget. Both models use a sequence length of 8192, 512 training anchors per sequence, a global batch size of 512, and a fixed 5B-token training subset repeated for 10 epochs. We evaluate the Drafter on GSM8K, MATH, HumanEval and MBPP using exact speculative verification with a proposal horizon of 16 draft tokens.

Following DSpark, we report mean accepted length (MAL), defined as

$$\text{MAL} = \frac{1}{R} \sum_{r=1}^R c_r, \quad (27)$$

where c_r is the number of tokens committed in verification round r , including the Target correction or bonus token when present.

Results: As shown in Table 9, concept conditioning improves MAL consistently across all four benchmarks. The macro-average MAL increases from 5.933 to 6.180, corresponding to a 4.17% relative improvement. The largest gain is observed on HumanEval, where MAL improves by 7.59%. These results indicate that the chunk-level concept representation complements token-level Target features with broader predictive context, improving the consistency of block-parallel proposals with negligible parameter overhead.

6. Related Work

6.1. Abstract-Level Prediction

A growing line of research predicts representations rather than reconstructing fine-grained inputs. Joint-embedding predictive architectures learn representations by predicting latent targets that retain semantic structure while discarding input details that are difficult to predict and unnecessary for the learned representation [Assran et al., 2023]. Large Concept Models apply abstract-level autoregressive modeling to language by mapping complete sentences into a shared continuous embedding space [Barrault et al., 2024]. These approaches motivate targets above raw input units, but use continuous prediction spaces that are typically defined by a separate encoder. Concept-level language modeling instead learns a higher-level prediction space within the language model while retaining token-level generation.

6.2. Hierarchical and Latent-Space Language Modeling

Hierarchical language models introduce a shorter high-level sequence over groups of fine-grained units. The Hourglass Transformer compresses and later upsamples token sequences [Nawrot et al., 2022], while MegaByte performs global modeling over fixed byte patches and local modeling within each patch [Yu et al., 2023]. BLT and H-Net replace fixed tokenization with data-dependent byte patches [Hwang et al., 2026, Pagnoni et al., 2025]. At the token level, ContextLM learns predictive context embeddings, and DLCM constructs higher-level units through dynamic chunking [Dai et al., 2025, Qu et al., 2025]. These methods primarily alter the computational granularity or the mechanism used to construct latent units.

Other work changes the prediction objective or introduces latent states into the generative process. Multi-token prediction adds auxiliary heads for several future tokens while retaining token-level targets [Gloeckle et al., 2024]. Methods based on latent reasoning or continuous concepts instead interleave or predict continuous hidden states [Hao et al., 2024, Su et al., 2025]. ConceptLM is the direct foundation of our work: it introduced NCP as a discrete concept-level objective, learned a product-quantized concept vocabulary jointly with the language model, and conditioned token generation on predicted concepts [Liu et al., 2026]. ConceptLM evaluated models trained from scratch up to 1.5B parameters and added NCP to an existing 8B model through continual pre-training. NCP-ArchPreview instead studies an 8.9B model derived from the Olmo-3-7B architecture in which NCP is active from the start of pre-training over several trillion tokens and during subsequent training stages.

6.3. Residual Connections in Deep Transformers

Standard residual connections accumulate cross-layer information into a single additive stream, motivating mechanisms that selectively reuse earlier representations. DenseFormer uses input-independent learned weights to average previous block outputs across depth [Pagliardini et al., 2024]. DeepCrossAttention uses input-dependent weights to combine layer outputs and additionally introduces depth-wise cross-attention [Heddes et al., 2025]. In parallel, MUDDFormer predicts token-conditioned dense-connection weights separately for the query, key, value, and residual streams [Xiao et al., 2025]. Attention Residuals instead applies softmax attention over preceding layer outputs, with a blockwise variant for scalable training [Team et al., 2026b]. Depth-Attention places cross-layer selection inside self-attention by mixing earlier value states through the existing QKV pathway [Zeng et al., 2026]. NCP-ArchPreview adopts MUDDFormer’s single-stream DD formulation as Intra-Module Residual Connections (IRC), enabling full-history reuse within each module. Cross-Module Residual Connections (CRC) further enable target-conditioned depth selection across the Encoder, Concept Module, and Decoder.

7. Limitations

Extension to long-context training: This report focuses on NCP-ArchPreview after 5.73T tokens of pre-training and 100B tokens of mid-training at standard context lengths. Long-context training is not included in the current architecture preview and remains part of our planned full training recipe. Since the concept-level pathway operates on a compressed sequence, longer contexts may provide a particularly favorable setting for latent-space modeling. Extending NCP-ArchPreview to this regime will allow us to study how concept prediction and hierarchical residual routing scale to substantially longer dependencies.

Relationship between language-modeling loss and downstream performance: NCP-ArchPreview maintains a clear token-level language-modeling loss advantage over OLMo-3-7B throughout both pre-training and mid-training, together with positive aggregate downstream gains at both checkpoints. The downstream improvement is more pronounced after pre-training, where the overall average increases by 2.45 points, than after mid-training, where the gain is 0.59 points and varies across benchmarks. Similar stage- and task-dependent relationships between cross-entropy loss and downstream capability have also been observed in recent work [Patel et al., 2026, Yano et al., 2026].

These results establish a consistent optimization advantage for NCP-ArchPreview while suggesting that the conversion from lower language-modeling loss to downstream performance depends on the training distribution and evaluation domain. Future work will explore additional mid-training recipes, capability-aware data selection, and checkpoint selection criteria that combine language-modeling loss with downstream and proxy evaluations.

8. Conclusion

This work establishes that latent representations learned by a language model can serve as first-class prediction targets at trillion-token scale. NCP-ArchPreview turns this idea into an 8.9B-parameter architecture that jointly models tokens and learned discrete concepts while preserving standard autoregressive generation. After pre-training on 5.73T tokens, it reaches the final loss of OLMo-3-7B with only 51.3% of its training tokens and improves the downstream macro-average by 2.45 points, including a 5.99-point gain on GSM8K. Controlled experiments show complementary gains from the latent architecture, hierarchical routing, and NCP; the complete model approaches a parameter-aligned 40-layer Transformer with only 85% of its computation, while scaling-law experiments indicate a 1.74 $\times$ improvement in compute efficiency. The learned concept space also supports efficient domain adaptation through only 17M VQ parameters and improves the mean accepted length of a DFlash2-based speculative drafter by 4.17% with negligible overhead. Together, these results provide the largest-scale validation of latent-space language modeling to date and position joint token and concept prediction as a practical, scalable blueprint for more capable and efficient language models. Extending this architecture through long-context training and translating its optimization advantages more consistently into downstream capability remain important directions for future work.

References

- Marianne Arriola, Aaron Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models, 2025. URL <https://arxiv.org/abs/2503.09573>.
- Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding

- predictive architecture. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629. IEEE, 2023.
- Mahmoud Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. URL <https://arxiv.org/abs/2108.07732>.
- Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research*, 2024.
- Loïc Barrault, Paul-Ambroise Duquenne, Maha Elbayad, Artyom Kozhevnikov, Belen Alastruey, Pierre Andrews, Mariano Coria, Guillaume Couairon, Marta R Costa-jussà, David Dale, et al. Large concept models: Language modeling in a sentence representation space. *arXiv preprint arXiv:2412.08821*, 2024.
- Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(05):7432–7439, 2020. doi: 10.1609/aaai.v34i05.6239. URL <https://ojs.aaai.org/index.php/AAAI/article/view/6239>.
- Andreas Blattmann, Robin Rombach, Huan Ling, Tim Dockhorn, Seung Wook Kim, Sanja Fidler, and Karsten Kreis. Align your latents: High-resolution video synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 22563–22575, 2023.
- Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q. Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. MultiPL-E: A scalable and extensible approach to benchmarking neural code generation. *arXiv preprint arXiv:2208.08227*, 2022. URL <https://arxiv.org/abs/2208.08227>.
- Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding, 2026. URL <https://arxiv.org/abs/2602.06036>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL <https://arxiv.org/abs/2107.03374>.
-

- Daixuan Cheng, Shaohan Huang, and Furu Wei. Adapting large language models via reading comprehension. In *International Conference on Learning Representations*, volume 2024, pages 48624–48652, 2024.
- Shuang Cheng, Yihan Bian, Dawei Liu, Yuhua Jiang, Yihao Liu, Linfeng Zhang, Qian Yao, Zhongbo Tian, Wenhai Wang, Qipeng Guo, Kai Chen, Bqing Qi, and Bowen Zhou. SDAR: A synergistic diffusion-AutoRegression paradigm for scalable sequence generation. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 22058–22075, San Diego, California, United States, July 2026a. Association for Computational Linguistics. doi: 10.18653/v1/2026.findings-acl.1110. URL <https://aclanthology.org/2026.findings-acl.1110/>.
- Xin Cheng, Xingkai Yu, Chenze Shao, Jiashi Li, Yunfan Xiong, Yi Qian, Jiaqi Zhu, Shirong Ma, Xiaokang Zhang, Jiasheng Ye, Qinyu Chen, Chengqi Deng, Jiping Yu, Damai Dai, Zhengyan Zhang, Yixuan Wei, Yixuan Tan, Wenkai Yang, Runxin Xu, Yu Wu, Zhean Xu, Xuanyu Wang, Muyang Chen, Rui Tian, Xiao Bi, Zhewen Hao, Shaoyuan Chen, Huanqi Cao, Wentao Zhang, Anyi Xu, Huishuai Zhang, Dongyan Zhao, and Wenfeng Liang. DSpark: Confidence-scheduled speculative decoding with semi-autoregressive generation, 2026b. URL <https://arxiv.org/abs/2607.05147>.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018. URL <https://arxiv.org/abs/1803.05457>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Beiya Dai, Yuliang Liu, Daozheng Xue, Yunchong Song, Qipeng Guo, Kai Chen, Xinbing Wang, Bowen Zhou, and Zhouhan Lin. Context-level language modeling by learning predictive context embeddings. *arXiv preprint arXiv:2510.20280*, 2025.
- Damai Dai, Li Dong, Yaru Hao, Zhifang Sui, Baobao Chang, and Furu Wei. Knowledge neurons in pre-trained transformers. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8493–8502, 2022.
- DeepSeek-AI. Deepseek llm: Scaling open-source language models with longtermism, 2024. URL <https://arxiv.org/abs/2401.02954>.
- DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu,

- Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. Deepseek-v3 technical report, 2025. URL <https://arxiv.org/abs/2412.19437>.
- Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2368–2378. Association for Computational Linguistics, 2019. doi: 10.18653/v1/N19-1246. URL <https://aclanthology.org/N19-1246/>.
- Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 conference on empirical methods in natural language processing*, pages 5484–5495, 2021.
- Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*, 2024.
- Robert Gray. Vector quantization. *IEEE Assp Magazine*, 1(2):4–29, 1984.
- Wes Gurnee and Max Tegmark. Language models represent space and time. In *The Twelfth International Conference on Learning Representations*, 2024.
- Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.
- Mike Heddes, Adel Javanmard, Kyriakos Axiotis, Gang Fu, MohammadHossein Bateni, and Vahab Mirrokni. Deepcrossattention: Supercharging transformer residual connections. *arXiv preprint arXiv:2502.06785*, 2025.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021a. URL <https://arxiv.org/abs/2009.03300>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021b. URL <https://openreview.net/forum?id=7Bywt2mQsCe>.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

- Sukjun Hwang, Brandon Wang, and Albert Gu. Dynamic chunking for end-to-end hierarchical sequence modeling. In *International Conference on Learning Representations*, volume 2026, pages 149273–149313, 2026.
- Inco AI. DFlash 2: Keep drafting parallel. <https://inco.ai/blog/dflash2/>, August 2026. Technical blog.
- Herve Jegou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence*, 33(1):117–128, 2010.
- Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences*, 11(14):6421, 2021. doi: 10.3390/app11146421. URL <https://www.mdpi.com/2076-3417/11/14/6421>.
- Keller Jordan, Yuchen Jin, Vlado Boza, You Jiacheng, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon/>.
- Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611, 2017.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13): 3521–3526, 2017.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL <https://aclanthology.org/Q19-1026/>.
- Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-Tau Yih, Daniel Fried, Sida Wang, and Tao Yu. DS-1000: A natural and reliable benchmark for data science code generation. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 18319–18345. PMLR, 2023. URL <https://proceedings.mlr.press/v202/lai23b.html>.
- Yann LeCun et al. A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Review*, 62(1):1–62, 2022.
- Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models. *arXiv preprint arXiv:2206.14858*, 2022. URL <https://arxiv.org/abs/2206.14858>.
- Kenneth Li, Aspen K. Hopkins, David Bau, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Emergent world representations: Exploring a sequence model trained on a synthetic task. In *International Conference on Learning Representations*, 2023.

- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023. URL <https://arxiv.org/abs/2305.20050>.
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.52202/075280-0943. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract.html.
- Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is scalable for llm training, 2025. URL <https://arxiv.org/abs/2502.16982>.
- Yuliang Liu, Yunchong Song, Yixuan Wang, Kewen Ge, Alex Lamb, Qipeng Guo, Kai Chen, Bowen Zhou, and Zhouhan Lin. Next concept prediction in discrete latent space leads to stronger language models. *arXiv preprint arXiv:2602.08984*, 2026.
- Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2022.
- Seyed Iman Mirzadeh, Keivan Alizadeh, Hooman Shahrokhi, Oncl Tuzel, Samy Bengio, and Mehrdad Farajtabar. GSM-symbolic: Understanding the limitations of mathematical reasoning in large language models. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=AjXkRZlvjB>.
- Arindam Mitra, Hamed Khanpour, Corby Rosset, and Ahmed Awadallah. Orca-math: Unlocking the potential of slms in grade school math. *arXiv preprint arXiv:2402.14830*, 2024.
- Piotr Nawrot, Szymon Tworowski, Michał Tyrolski, Łukasz Kaiser, Yuhuai Wu, Christian Szegedy, and Henryk Michalewski. Hierarchical transformers are more efficient language models. In *Findings of the Association for Computational Linguistics: NAACL 2022*, pages 1559–1571, 2022.
- Team Olmo, :, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heineman, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, Jacob Morrison, Jake Poznanski, Kyle Lo, Luca Soldaini, Matt Jordan, Mayee Chen, Michael Noukhovitch, Nathan Lambert, Pete Walsh, Pradeep Dasigi, Robert Berry, Saumya Malik, Saurabh Shah, Scott Geng, Shane Arora, Shashank Gupta, Taira Anderson, Teng Xiao, Tyler Murray, Tyler Romero, Victoria Graf, Akari Asai, Akshita Bhagia, Alexander Wettig, Alisa Liu, Aman Rangapur, Chloe Anastasiades, Costa Huang, Dustin Schwenk, Harsh Trivedi, Ian Magnusson, Jaron Lochner, Jiacheng Liu, Lester James V. Miranda, Maarten Sap, Malia Morgan, Michael Schmitz, Michal Guerquin, Michael Wilson, Regan Huff, Ronan Le Bras, Rui Xin, Rulin Shao, Sam Skjonsberg, Shannon Zejiang Shen, Shuyue Stella Li, Tucker Wilde, Valentina Pyatkin, Will Merrill, Yapei Chang, Yuling Gu, Zhiyuan Zeng, Ashish Sabharwal, Luke Zettlemoyer, Pang Wei Koh, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. Olmo 3, 2026. URL <https://arxiv.org/abs/2512.13961>.
- Matteo Pagliardini, Amirkeivan Mohtashami, Francois Fleuret, and Martin Jaggi. Denseformer: Enhancing information flow in transformers via depth weighted averaging. *Advances in neural information processing systems*, 37:136479–136508, 2024.

- Artidoro Pagnoni, Ramakanth Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason E Weston, Luke Zettlemoyer, et al. Byte latent transformer: Patches scale better than tokens. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9238–9258, 2025.
- Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. MedMCQA: A large-scale multi-subject multi-choice dataset for medical domain question answering. In *Proceedings of the Conference on Health, Inference, and Learning*, volume 174 of *Proceedings of Machine Learning Research*, pages 248–260. PMLR, 2022. URL <https://proceedings.mlr.press/v174/pal22a.html>.
- Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1525–1534. Association for Computational Linguistics, 2016. doi: 10.18653/v1/P16-1144. URL <https://aclanthology.org/P16-1144/>.
- Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 39643–39666. PMLR, 2024.
- Arkil Patel, Siva Reddy, Marius Mosbach, and Dzmitry Bahdanau. Forecasting downstream performance of llms with proxy metrics, 2026. URL <https://arxiv.org/abs/2605.18607>.
- Xingwei Qu, Shaowen Wang, Zihao Huang, Kai Hua, Fan Yin, Rui-Jie Zhu, Jundong Zhou, Qiyang Min, Zihao Wang, Yizhi Li, et al. Dynamic large concept models: Latent reasoning in an adaptive semantic space. *arXiv preprint arXiv:2512.24617*, 2025.
- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392. Association for Computational Linguistics, 2016. doi: 10.18653/v1/D16-1264. URL <https://aclanthology.org/D16-1264/>.
- Siva Reddy, Danqi Chen, and Christopher D. Manning. CoQA: A conversational question answering challenge. *Transactions of the Association for Computational Linguistics*, 7:249–266, 2019. doi: 10.1162/tacl_a_00266. URL <https://aclanthology.org/Q19-1016/>.
- Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10684–10695, 2022.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial winograd schema challenge at scale. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34 (05):8732–8740, 2020. doi: 10.1609/aaai.v34i05.6399. URL <https://ojs.aaai.org/index.php/AAAI/article/view/6399>.
- Maarten Sap, Hannah Rashkin, Derek Chen, Ronan Le Bras, and Yejin Choi. Social IQa: Commonsense reasoning about social interactions. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 4463–4473. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1454. URL <https://aclanthology.org/D19-1454/>.

soldni. Jeopardy questions from mosaic gauntlet. Hugging Face dataset, 2024. URL <https://huggingface.co/datasets/soldni/jeopardy>. Revision ae-bec90f52498628fc26f7c06c00cd618374026f; MosaicML Gauntlet subset.

DiJia Su, Hanlin Zhu, Yingchen Xu, Jiantao Jiao, Yuandong Tian, and Qinqing Zheng. Token assorted: Mixing latent and text tokens for improved language model reasoning. *arXiv preprint arXiv:2502.03275*, 2025.

Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. CommonsenseQA: A question answering challenge targeting commonsense knowledge. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4149–4158. Association for Computational Linguistics, 2019. doi: 10.18653/v1/N19-1421. URL <https://aclanthology.org/N19-1421/>.

Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, M. C., Jianfeng Cai, Xinyuan Cai, Peizhou Cao, Yuxuan Cao, Ziwei Chai, Y. Charles, H. S. Che, Guanduo Chen, Guangyu Chen, Guanzheng Chen, Huarong Chen, Jia Chen, Jianlong Chen, Jun Chen, Kexin Chen, Peng Chen, Ruijue Chen, Wentao Chen, Xin Chen, Yang Chen, Yanru Chen, Yifei Chen, Yingjiang Chen, Yuankun Chen, Yujie Chen, Yutian Chen, Zhirong Chen, Dazhi Cheng, Yean Cheng, Jialei Cui, Jingbing Cui, Anqi Dai, Jiaqi Deng, Hao Ding, Rui Ding, Shaofeng Ding, Mengfan Dong, Mengnan Dong, Yuhao Dong, Yuxin Dong, Angang Du, Chenzhuang Du, Dikang Du, Jusen Du, Yulun Du, Yu Fan, Jing Feng, Qiulin Feng, Yichen Feng, Kelin Fu, Qiang Fu, Fuxuan Gao, Hongcheng Gao, Jingyue Gao, Tong Gao, Weijia Gao, Shangyi Geng, Jie Gong, Linhu Gong, Shengao Gong, Xiaochen Gong, Qizheng Gu, Yicheng Gu, Shuhao Guan, Haiqing Guo, Shiqi Guo, Xiang Guo, Zhengyan Guo, Beixi Hao, Wenxin Hao, Xiaoru Hao, Dailan He, Haotian He, Lehan He, Qi He, Weiran He, Xinran He, Xinyi He, Yibo He, Yunjia He, Chao Hong, Tiange Hong, Hao Hu, Jiayi Hu, Ruikun Hu, Weiming Hu, Yangyang Hu, Zhenxing Hu, Liang Hua, Jinbin Huang, Ke Huang, Ruiyuan Huang, Siying Huang, Weixiao Huang, Yan Huang, Zhengjie Huang, Zhiqi Huang, Yulong Hui, Chaobo Jia, Yutong Jiang, Zhejun Jiang, Zuoyou Jiang, Wenyi Jin, Xinyi Jin, Yu Jing, Huanjun Kong, Guokun Lai, Aidi Li, Cheng Li, Chengyuan Li, Cong Li, Fang Li, Guanyu Li, Haoyang Li, Jia Li, Junxiong Li, Lei Li, Letian Li, Lincan Li, Weihong Li, Wentao Li, Xintong Li, Yang Li, Yishen Li, Yiwei Li, Yuxiao Li, Zhaowei Li, Zhaoxi Li, Zheming Li, Zhengxiao Li, Zhiyuan Li, Jiawei Lin, Xiaohan Lin, Yibo Lin, Zichao Lin, Ziyan Lin, Bill Liu, Boxiao Liu, Chuan Liu, Liang Liu, Shaowei Liu, Shudong Liu, Shuran Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yanming Liu, Yibo Liu, Yipeng Liu, Zhengying Liu, Zhiheng Liu, Enzhe Lu, Haoyu Lu, Linqiang Lu, Tingzhan Lu, Zhiyuan Lu, Aotian Luo, G. Luo, Junyu Luo, Yifan Luo, B. Lyu, Wenzhou Lyu, Shaoguang Mao, Yuan Mei, Xin Men, Mingqing Ni, Yixuan Niu, Siyuan Pan, Shujun Peng, Zhangyang Qi, Ruoyu Qin, ZeChao Qin, Zeyu Qin, Haiquan Qiu, Jianxin Qiu, Jiezhong Qiu, Bowen Qu, Yuhao Qu, Zeyu Shang, Youbo Shao, Han Shen, Jincheng Shi, Juanfeng Shi, Lidong Shi, Shengyuan Shi, Wingchun Siu, Pengwei Song, Xiaoxi Song, Jianlin Su, Yunfeng Su, Zhaochen Su, Lin Sui, Jingsong Sun, Junyao Sun, Shaoning Sun, Shuzhe Sun, Tongyu Sun, Yujun Sun, Yunpeng Tai, Chuning Tang, Heyi Tang, Sirui Tang, Zecheng Tang, Chaoran Tian, Rongpeng Tian, Yu Tian, Wei Tu, Chensi Wang, Chuang Wang, Chunjie Wang, Dinglu Wang, Feng Wang, Hailong Wang, Haiming Wang, Hao Wang, Hao Wang, Huaqing Wang, Hui Wang, Jiayi Wang, Jinglong Wang, Jinhong Wang, Jiuzheng Wang, Linian Wang, Shaobo Wang, Shenchi Wang, Shuyi Wang, Si Wang, Siyuan Wang, Tianfu Wang, Wenjue Wang, Xingran Wang, Xinmei Wang, Xinyuan Wang, Xusheng Wang, Yalin Wang, Yangkun Wang, Yao Wang, Yaoyu Wang, Yejie Wang, Yiqin Wang, Yucheng Wang, Yuzhi Wang, Zhaoji Wang, Zhaowei Wang, Zhengtao Wang, Zhenhao Wang, Zhongsheng Wang, Zifan Wang, Chu Wei, Ming Wei, Shouxin Wei, Zichen Wen, Fan Wu, Haoning Wu, Rucong Wu, Wenhao Wu, Xiaoxue Wu, Yingcong Wu, Yongqi Wu, Yuxin Wu, Zijian Wu, Xinglang Xian, Chenxuan Xiang, Yuye Xiang, Bocheng Xiao, Chenjun Xiao, Xin Xiao, Jin Xie,

- Xiaotong Xie, Yifeng Xie, Zhe Xie, Bowei Xing, Yiming Xiong, Baosheng Xu, Boyu Xu, Jiale Xu, Jianfan Xu, Jing Xu, Jinjing Xu, L. H. Xu, Qingtao Xu, Shuyao Xu, Suting Xu, Tiantian Xu, Tianxiang Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Ye Xu, Yueni Xu, Ziyao Xu, Haonan Xue, Junjie Yan, Yaoyao Yan, Fan Yang, Guangyao Yang, Hao Yang, Junwei Yang, Ruoyu Yang, Wenjie Yang, Xiaofei Yang, Xinyu Yang, Yi Yang, Yiling Yang, Ying Yang, Yuchen Yang, Zhen Yang, Zhilin Yang, Zian Yang, Zuhao Yang, Haotian Yao, Dan Ye, Haoran Ye, Wenjie Ye, Zhanbo Ye, Bohong Yin, Haoxiang Yin, Xietong Yin, Chengzhen Yu, Haozhen Yu, Longhui Yu, Shengnan Yu, Shuying Yu, Tianxiang Yu, Enming Yuan, Mengjie Yuan, Tongtian Yue, Wei Yue, Yang Yue, Dunyuan Zha, Haobing Zhan, B. H. Zhang, Dehao Zhang, Fei Zhang, Hao Zhang, Haoyuan Zhang, Huanyu Zhang, Jiapei Zhang, Jiaxuan Zhang, Jin Zhang, Kaiyi Zhang, Miaozen Zhang, Puqi Zhang, Qinglei Zhang, Rong Zhang, Rui Zhang, Shaoshuai Zhang, Shiyi Zhang, Xiaobin Zhang, Xiaoyun Zhang, Y. Zhang, Yangkun Zhang, Ye Zhang, Yichi Zhang, Yikun Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Zijing Zhang, Bin Zhao, Chenguang Zhao, Feifan Zhao, Jinglun Zhao, Jinxiang Zhao, Shuai Zhao, Wenshuo Zhao, Xiangyu Zhao, Xuanle Zhao, Yikai Zhao, Zijia Zhao, Haozhi Zheng, Huabin Zheng, Ruihan Zheng, Shaojie Zheng, Tengyang Zheng, Haofeng Zhong, Lei Zhong, Longguang Zhong, M. Zhou, Qianqiang Zhou, Runjie Zhou, Ruozhang Zhou, Xinyu Zhou, Yiqiao Zhou, Zaida Zhou, Jinguo Zhu, Liya Zhu, Xinhao Zhu, Yangjunfeng Zhu, Yuxuan Zhu, Zhen Zhu, Chen Zhuang, Weiyu Zhuang, and Xinxing Zu. Kimi k3: Open frontier intelligence, 2026a. URL <https://arxiv.org/abs/2607.24653>.
- Kimi Team, Guangyu Chen, Yu Zhang, Jianlin Su, Weixin Xu, Siyuan Pan, Yaoyu Wang, Yucheng Wang, Guanduo Chen, Bohong Yin, et al. Attention residuals. *arXiv preprint arXiv:2603.15031*, 2026b.
- Gido M Van de Ven, Nicholas Soures, and Dhireesha Kudithipudi. Continual learning and catastrophic forgetting. *arXiv preprint arXiv:2403.05175*, 2024.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhui Chen. MMLU-pro: A more robust and challenging multi-task language understanding benchmark. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-3018. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/ad236edc564f3e3156e1b2feafb99a24-Abstract-Datasets_and_Benchmarks_Track.html.
- Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. Magicoder: Empowering code generation with oss-instruct. *arXiv preprint arXiv:2312.02120*, 2023.
- Da Xiao, Qingye Meng, Shengping Li, and Xingyuan Yuan. Muddformer: Breaking residual bottlenecks in transformers via multiway dynamic dense connections. *arXiv preprint arXiv:2502.12170*, 2025.
- Kazuki Yano, Shun Kiyono, Sosuke Kobayashi, Sho Takase, and Jun Suzuki. Pre-training llm without learning rate decay enhances supervised fine-tuning, 2026. URL <https://arxiv.org/abs/2603.16127>.
- Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. Megabyte: Predicting million-byte sequences with multiscale transformers. *Advances in Neural Information Processing Systems*, 36:78808–78823, 2023.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/P19-1472. URL <https://aclanthology.org/P19-1472/>.

Boyi Zeng, Yiqin Hao, Zitong Wang, Shixiang Song, He Li, Feichen Song, Yifan Liu, Ziwei He, Xinbing Wang, and Zhouhan Lin. Depth-attention: Cross-layer value mixing for language models. *arXiv preprint arXiv:2606.05014*, 2026.

Jiebin Zhang, Zhenghan Yu, Song Liu, Eugene J. Yu, Zheng Li, Dawei Zhu, Jiangshan Duo, Weimin Xiong, Yifan Song, Guanghua Yu, Jianchen Zhu, and Sujian Li. DFlare: Scaling up draft capacity for block diffusion speculative decoding, 2026. URL <https://arxiv.org/abs/2606.02091>.

Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen Gong, Thong Hoang, Armel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, Binyuan Hui, Niklas Muennighoff, David Lo, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro Von Werra. BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=YrycTjllL0>.

A. Author List

A.1. Full List (Alphabetically)

Jiaqi Cao; Chiyu Chen; Shuang Cheng; Xu Cheng; Beiya Dai; Yufan Feng; Kewen Ge; Ruijun Ge; Jiayi Huang; Yang Jiao; Dahua Lin; Zhouhan Lin; Yifan Liu; Yuliang Liu; Biqing Qi; Mowen Ruan; Junzhe Shen; Yunchong Song; Hao Sun; Zhongbo Tian; Yixuan Wang; Rubin Wei; Jiaxin Xiong; Kangyu Yang; Qian Yao; Qi Zhang; Bowen Zhou

A.2. Contributions

Architecture Design	Zhouhan Lin; Yuliang Liu
Architecture Optimization	Yuliang Liu; Jiayi Huang; Jiaxin Xiong; Beiya Dai
Model Training	Yuliang Liu; Jiayi Huang; Yang Jiao; Junzhe Shen; Rubin Wei; Xu Cheng; Yixuan Wang
Scaling Ladder	Mowen Ruan; Yuliang Liu
Ablation and Analysis	Jiayi Huang; Rubin Wei; Yufan Feng; Yuliang Liu
Inference and Deployment	Yixuan Wang; Shuang Cheng; Biqing Qi; Qian Yao; Kewen Ge
Evaluation	Yixuan Wang; Yuliang Liu
Data Analysis	Hao Sun; Yunchong Song; Yifan Liu; Yuliang Liu
Infrastructure	Ruijun Ge; Chiyu Chen; Zhongbo Tian
Project Leader	Zhouhan Lin
Project Advisors	Zhouhan Lin; Qi Zhang; Dahua Lin; Bowen Zhou

✉ Correspondence to: liuyl03181@gmail.com | hantek@sjtu.edu.cn

B. Additional Architecture Details

B.1. Complete Architecture Configuration

Table 10 summarizes the complete architecture configuration of NCP-ArchPreview.

B.2. Forward-Pass Pseudocode

Algorithm 1 summarizes the end-to-end training forward pass. The intra-module residual connections (IRC) and cross-module residual connections (CRC) follow Eqs. 14–20 in Section 2.5. The causal alignment between concept-level and token-level states follows Eq. 11.

For compactness, S^e and S^c denote the intermediate layer states exported by the Token Encoder and Concept Module, respectively. The operator ChunkPool_k converts token-level residual states to concept resolution, whereas $\text{CausalShiftAndRepeat}$ converts concept-level states back to token resolution without exposing future concepts.

C. Analysis of OLMo-3 Mid-Training Data with Proxy Metrics

The preceding sections describe the model architecture and training setup. We next examine whether candidate data recipes for the second training stage can be compared without repeatedly executing the full downstream evaluation suite. To this end, we evaluate three matched-budget Stage-2 recipes

Table 10 | Architecture configuration of NCP-ArchPreview.

Component	Configuration
Model class	Latent-space autoregressive language model
Total parameters	8.94B
Token Encoder	16 causal Transformer layers
Concept Module	8 causal Transformer layers
Token Decoder	16 causal Transformer layers
Hidden dimension	4,096
FFN dimension	11,008
Attention heads	32
Key-value groups	32
Attention-head dimension	128
Vocabulary size	100,278
Maximum training length	8,192 tokens
Position encoding	RoPE with base 500,000
Token-level local-attention window	4,096 tokens
Full-attention pattern	Every fourth layer uses full attention
Activation	SwiGLU
Normalization	RMSNorm with $\epsilon = 10^{-6}$
Attention normalization	Layer-wise QK RMSNorm
Attention and hidden dropout	0
Concept compression factor k	4 token states per concept
Concept sequence length	$M = T/k$ after padding and masking
PQ segments S	32
Codewords per segment N	128
Codeword dimension	128
Latent vocabulary capacity	$N^S = 128^{32}$ possible code combinations
Token-level objective	Next-token prediction (NTP)
Latent-space objective	Next-concept prediction (NCP)
Concept predictor	Dedicated Concept Module
Concept feedback	Causally shifted, repeated k times, and added to token states
Intra-module residual routing	IRC in the Token Encoder, Concept Module, and Token Decoder
Cross-module residual routing	Token Encoder→Concept Module, Token Encoder→Token Decoder, and Concept Module→Token Decoder
Parameter precision	BF16

with a fixed, capability-specific held-out set. The Stage-1 checkpoint is included as a cross-stage reference, while recipe selection is based on comparisons among the three Stage-2 variants.

C.1. Held-out Evaluation and Scoring

The expanded held-out set is constructed from 177,202 expert trajectories spanning 63 sources and six domains, corresponding to approximately 100M teacher-suffix tokens. All trajectories are produced by a single frozen Qwen3.7-Plus teacher under a common generation protocol. The prompt, expert suffix, source identity, and capability annotation are fixed before candidate checkpoints are evaluated, ensuring that every recipe is compared on an identical scoring view. Invalid generations are excluded, and a more restrictive trusted subset is retained for sensitivity analysis.

Algorithm 1 End-to-end training of NCP-ArchPreview with hierarchical residual routing

Input: Tokens $x_{1:T}$; compression factor $k = 4$; PQ codebooks $\mathcal{E}^s = \{\mathbf{e}_n^s\}_{n=1}^N, s = 1, \dots, S$

Input: Loss weights α and β

- 1: $M \leftarrow \lfloor T/k \rfloor$
- 2: $(\mathbf{H}, \mathbf{S}^e) \leftarrow \text{TokenEncoder}_{\theta_e}^{\text{IRC}}(x_{1:T})$ ▷ IRC inside the TokenEncoder
- 3: $\mathbf{C} \leftarrow \text{MeanPool}_k(\mathbf{H})$
- 4: $\overline{\mathbf{S}}^e \leftarrow \text{ChunkPool}_k(\mathbf{S}^e)$ ▷ TokenEncoder→ConceptModule CRC alignment
- 5: **for** $m = 1, \dots, M$ **do**
- 6: Split $\mathbf{c}_m$ into $\mathbf{c}_m^1, \dots, \mathbf{c}_m^S$
- 7: **for** $s = 1, \dots, S$ **do**
- 8: $n_m^s \leftarrow \arg \min_{n \in \{1, \dots, N\}} \|\mathbf{c}_m^s - \mathbf{e}_n^s\|_2^2$
- 9: $\mathbf{d}_m^s \leftarrow \mathbf{e}_{n_m^s}^s$
- 10: **end for**
- 11: **end for**
- 12: $\mathcal{L}_{\text{VQ}} \leftarrow \frac{1}{MS} \sum_{m=1}^M \sum_{s=1}^S \|\text{sg}[\mathbf{c}_m^s] - \mathbf{d}_m^s\|_2^2$
- 13: $(\mathbf{U}, \mathbf{S}^c) \leftarrow \text{ConceptModule}_{\theta_c}^{\text{IRC+CRC}}(\mathbf{C}_{1:M-1}; \overline{\mathbf{S}}^e)$ ▷ IRC and TokenEncoder→ConceptModule CRC
- 14: **for** $m = 2, \dots, M$ **do**
- 15: **for** $s = 1, \dots, S$ **do**
- 16: $\pi_m^s \leftarrow \text{softmax}(\text{PredictionHead}_s(\mathbf{u}_{m-1}))$
- 17: $\hat{\mathbf{c}}_m^s \leftarrow \sum_{n=1}^N \pi_{m,n}^s \mathbf{e}_n^s$
- 18: **end for**
- 19: $\hat{\mathbf{c}}_m \leftarrow \text{concat}(\hat{\mathbf{c}}_m^1, \dots, \hat{\mathbf{c}}_m^S)$
- 20: **end for**
- 21: $\mathcal{L}_{\text{NCP}} \leftarrow \frac{1}{(M-1)S} \sum_{m=2}^M \sum_{s=1}^S \|\hat{\mathbf{c}}_m^s - \text{sg}[\mathbf{c}_m^s]\|_2^2$
- 22: $\mathbf{B} \leftarrow \text{CausalShiftAndRepeat}((\hat{\mathbf{c}}_2, \dots, \hat{\mathbf{c}}_M), k, T)$ ▷ Direct predicted-concept feedback
- 23: $\tilde{\mathbf{H}} \leftarrow \mathbf{H} + \mathbf{B}$
- 24: $\mathbf{S}^{c \rightarrow d} \leftarrow \text{CausalShiftAndRepeat}(\mathbf{S}^c, k, T)$ ▷ ConceptModule→TokenDecoder CRC alignment
- 25: $\mathbf{H}^{\text{dec}} \leftarrow \text{TokenDecoder}_{\theta_d}^{\text{IRC+CRC}}(\tilde{\mathbf{H}}; \mathbf{S}^e, \mathbf{S}^{c \rightarrow d})$ ▷ IRC and two incoming CRC paths
- 26: $\ell \leftarrow \text{LMHead}(\mathbf{H}^{\text{dec}})$
- 27: $\mathcal{L}_{\text{NTP}} \leftarrow \text{CE}(\ell_{1:T-1}, x_{2:T})$
- 28: **return** $\mathcal{L}_{\text{NTP}} + \alpha \mathcal{L}_{\text{NCP}} + \beta \mathcal{L}_{\text{VQ}}$

For free-form tasks, we teacher-force the expert suffix and compute mean target-token negative log-likelihood,

$$\ell_m(i) = -\frac{1}{|T_i|} \sum_{j \in T_i} \log p_m(y_{ij} \mid x_i, y_{i,<j}) . \quad (28)$$

Item-level scores are first averaged within each source, after which source-level means are averaged within each capability leaf. This source-balanced aggregation prevents large benchmarks from dominating solely through their sample counts. Function generation, competition mathematics, academic STEM, and logical reasoning are retained as separate measurements rather than collapsed into a single cross-domain score. For multiple-choice tasks, we use a task-native contrastive statistic: on each HellaSwag item, the character-normalized log-likelihood of the correct continuation is compared with that of the strongest distractor. Both trajectory NLL and choice margin therefore require only batched likelihood evaluation, without autoregressive answer generation or an auxiliary judge model.

C.2. Recipe-level Results

Downstream results are obtained under a unified evaluation protocol. The Stage-1 model uses the standard pre-training evaluation configuration and is identified by $\text{GSM8K} = 46.78$ and $\text{MATH-500} = 14.34$. The three Stage-2 variants (V1/V2/V3) share the same inference and sampling configuration, making their relative performance the primary basis for recipe comparison. Stage-1 is used only to provide cross-stage context. In addition to rank agreement, we fit least-squares lines to assess whether differences in Proxy measurements reflect the magnitude of downstream performance differences. Table 11 reports the complete measurements and downstream results used in this analysis.

Table 11 | Exact inputs to the recipe-screening comparison. Panel (a) reports source-balanced trajectory NLL for the four free-form capability leaves and the task-native correct-versus-best-distractor margin for HellaSwag. Panel (b) reports the corresponding frozen downstream scores. The common prefix in the first header row combines with each Stage suffix to form the exact checkpoint ID. Bold marks the best value among the matched-budget Stage-2 recipes; the Stage-1 column provides cross-stage context and is not part of that selection.

Metric	NCP_ArchPreview_dolma3_8.9B_			
	Stage1_V1	Stage2_V1	Stage2_V2	Stage2_V3
<i>(a) Expanded 100M Proxy measurements</i>				
Function-generation NLL ↓	0.9338	0.7330	0.7501	0.7598
Competition-math NLL ↓	0.7979	0.5742	0.5949	0.6075
Academic-STEM NLL ↓	1.0570	0.8909	0.9007	0.9058
Logical-reasoning NLL ↓	0.9692	0.8148	0.8314	0.8355
HellaSwag margin ↑	0.09610	0.09678	0.09932	0.10083
<i>(b) Frozen downstream performance (%)</i>				
HumanEval ↑	31.27	45.60	42.19	39.96
MBPP ↑	36.09	50.91	49.34	46.60
MATH-500 ↑	14.34	43.74	41.66	37.21
Minerva ↑	15.13	42.20	40.39	36.70
MMLU-STEM ↑	55.88	61.84	59.85	57.72
BBH ↑	52.41	63.23	62.90	60.44
HellaSwag ↑	65.85	66.40	67.30	67.25

Across all four free-form capability leaves, the Stage-2 variants exhibit a consistent ordering: V1 achieves the lowest NLL, followed by V2 and V3. The corresponding downstream benchmarks show the same ordering: HumanEval and MBPP for code, MATH-500 and Minerva for mathematics, MMLU-STEM for academic STEM, and BBH for logical reasoning. Thus, each capability–benchmark mapping is concordant across all three pairwise Stage-2 comparisons. Relative to V3, V1 reduces NLL by 3.5% for function generation, 5.5% for competition mathematics, 1.6% for academic STEM, and 2.5% for logical reasoning. These reductions correspond to downstream improvements of 5.64 points on HumanEval, 4.31 on MBPP, 6.52 on MATH-500, 5.50 on Minerva, 4.11 on MMLU-STEM, and 2.79 on BBH. The same four Proxy orderings are preserved when the analysis is repeated on the trusted subset.

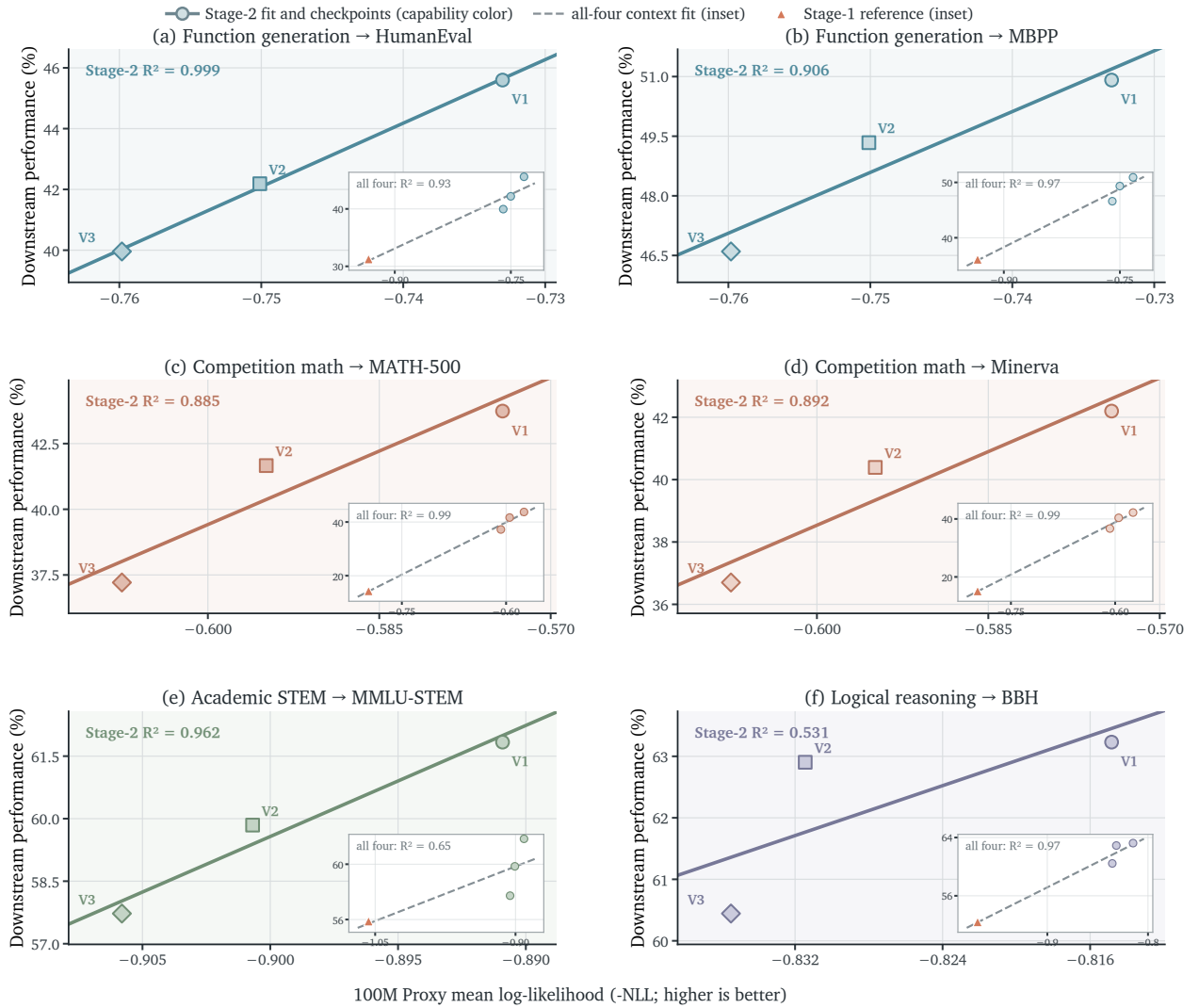


Figure 9 | Relationship between the expanded 100M Proxy and downstream benchmark performance. The main panels show the three same-protocol Stage-2 recipes. The horizontal axis is source-balanced mean log-likelihood ($-NLL$, higher is better), and the vertical axis is downstream performance in percent. Insets add the Stage-1 reference and the corresponding four-point fit for context. Each capability–benchmark mapping is reported separately; the lines are descriptive fits rather than calibrated predictors.

C.3. Interpretation and Scope

The fitted relationships in Figure 9 complement the rank comparison by characterizing the relative magnitudes of the observed differences. The Stage-2 R^2 values are 0.999 for HumanEval, 0.906 for MBPP, 0.885 for MATH-500, 0.892 for Minerva, and 0.962 for MMLU-STEM. BBH exhibits a lower R^2 of 0.531: its recipe ordering remains concordant, although the Proxy differences are not proportional to the downstream score differences. Taken together, the matched-budget comparisons identify V1 as the strongest of the three Stage-2 recipes across the evaluated free-form capabilities. This conclusion applies at the level of complete data recipes; it does not attribute the observed improvements to any individual source within a mixture.

HellaSwag further illustrates the importance of matching the scoring statistic to the task format. The

task-native choice margin increases from 0.09678 for V1 to 0.09932 for V2 and 0.10083 for V3. The downstream scores place V2 and V3 in the leading tier at 67.30 and 67.25, respectively, compared with 66.40 for V1. Because V2 and V3 differ by only 0.05 percentage points downstream, we treat them as effectively tied for recipe screening rather than infer a preference from this negligible gap. At this decision granularity, the choice margin distinguishes the leading V2/V3 tier from V1 and provides a more appropriate summary for multiple-choice evaluation.

In practice, a new mid-training recipe can be screened with a single frozen forward pass by comparing its capability-level NLL values and task-native margins with those of existing recipes. Retaining these measurements as a capability vector supports targeted recipe selection—for example, emphasizing code and mathematics—without imposing an arbitrary global aggregation across domains. The present evidence covers three Stage-2 recipes and one cross-stage reference. It therefore supports efficient recipe screening within the evaluated setting, but does not establish calibrated prediction for unseen model families or causal attribution to individual data components.

D. Scaling Experiments

This section describes how we conduct the scaling-ladder experiments. We first select several FLOPs budgets and design multiple allocations between model size and training tokens for each budget. Although the formula $C \approx 6N_{\text{param}}D$, where N_{param} denotes the number of non-embedding parameters, has been widely adopted as an approximation of training computation, it can be inaccurate for architectures with non-uniform layer activation. In NCP-ArchPreview, the Concept Module operates only once per chunk. We therefore compute the total training budget as

$$C = F_{\text{tok}}D,$$

where F_{tok} denotes the analytical training FLOPs per token and D denotes the number of training tokens. The FLOPs estimates exclude embedding parameters.

Previous study [DeepSeek-AI, 2024] finds that different experiments under different model/data allocations tend to have similar optimal hyperparameters at a fixed FLOPs budget. Motivated by this observation, we first choose a representative model at each FLOPs budget and perform search over learning rate and batch size on these models. After determining the optimal learning rate, we search for the best learning rate of the other models at the same FLOPs budget in a neighborhood around it. Each point in Figure 10 reports the result obtained using the best searched hyperparameters for the corresponding model and FLOPs budget. Table 12 lists the experimental settings for every point shown in the figure.

E. Evaluation Details

Benchmark coverage: The main comparison in Table 1 separates higher-is-better task scores from lower-is-better likelihood scores. The higher-is-better portion reports the following benchmark groups:

- **MMLU:** MMLU-Humanities, MMLU-Social Sciences, MMLU-STEM, MMLU-Other, and the aggregate MMLU score [Hendrycks et al., 2021a], together with MMLU-Pro [Wang et al., 2024].
- **Mathematical reasoning:** GSM8K [Cobbe et al., 2021], GSM-Symbolic [Mirzadeh et al., 2025], Minerva Math [Hendrycks et al., 2021b, Lewkowycz et al., 2022], and MATH-500 [Hendrycks et al., 2021b, Lightman et al., 2023].
- **Code generation:** BigCodeBench [Zhuo et al., 2025], HumanEval [Chen et al., 2021], DS-1000 [Lai et al., 2023], MBPP [Austin et al., 2021], and the HumanEval and MBPP variants of MultiPL-E [Cassano et al., 2022].

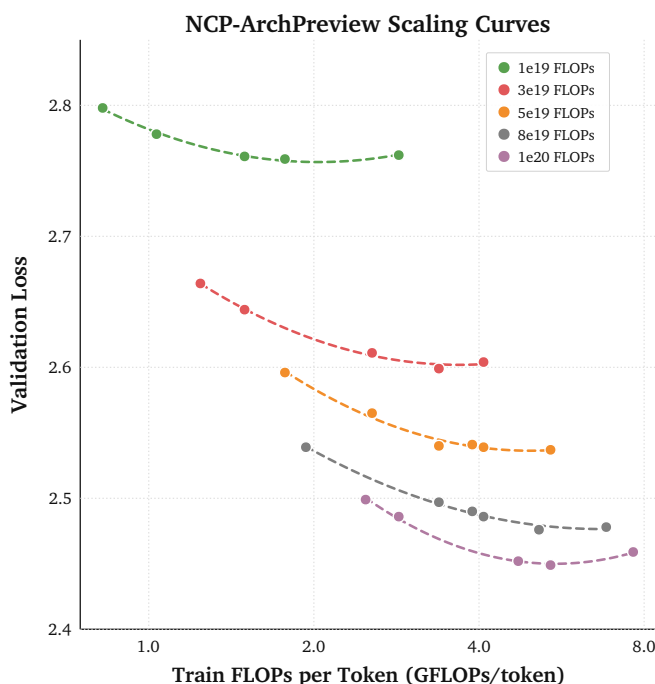


Figure 10 | NCP-ArchPreview IsoFLOP Curves. Markers denote experimental results, while dashed curves show the fitted trends through the measured points.

- **Multiple-choice STEM reasoning:** ARC-Easy and ARC-Challenge [Clark et al., 2018].
- **Multiple-choice non-STEM reasoning:** PIQA [Bisk et al., 2020], CommonsenseQA [Talmor et al., 2019], and SocialIQa [Sap et al., 2019].
- **General question answering:** HellaSwag [Zellers et al., 2019], WinoGrande [Sakaguchi et al., 2020], LAMBADA-Standard and LAMBADA-OpenAI [Paperno et al., 2016], MedMCQA [Pal et al., 2022], and MedQA [Jin et al., 2021].

The lower-is-better likelihood portion reports bits per byte (BPB) on HumanEval Gold [Chen et al., 2021], MBPP Gold [Austin et al., 2021], MT-MBPP Gold [Cassano et al., 2022], CoQA [Reddy et al., 2019], DROP [Dua et al., 2019], GSM8K Gold [Cobbe et al., 2021], Jeopardy [Soldni, 2024], LAMBADA-OpenAI [Paperno et al., 2016], Natural Questions [Kwiatkowski et al., 2019], and SQuAD [Rajpurkar et al., 2016]. The “Gold” suffix denotes evaluation of the likelihood assigned to a reference completion rather than free-form generation.

The group labels above are used only to organize the reported results and do not imply that all benchmarks within a group share the same evaluation format. In particular, the GenQA group contains both likelihood-ranked choice tasks and zero-shot language-modeling tasks.

We use a global random seed of 42 for few-shot example selection and generation. Prompt templates, demonstrations, answer choices, stop sequences, evaluation splits, and scoring implementations are held fixed across all evaluated models.

Multiple-choice and language-modeling accuracy evaluation. The four MMLU domain scores and the aggregate MMLU score are obtained using 5-shot multiple-choice evaluation on the test split. Each domain score is macro-averaged equally across the MMLU subjects assigned to that domain, while the aggregate MMLU score is macro-averaged equally across all 57 subjects. MMLU-Pro is evaluated using its 5-shot multiple-choice formulation.

Table 12 | NCP-ArchPreview scaling-ladder experiment settings and validation losses. Here, H is the hidden width, and $L_e/L_c/L_d$ are the encoder/Concept Module/decoder depths, F_{tok} is the analytical training FLOPs per token, and D is the training-token count. The reported loss is the lowest validation loss among the searched learning rates for each point.

C (FLOPs)	Pt.	Heads	H	$L_e/L_c/L_d$	F_{tok} (GF/token)	D (B)	Val. loss
1e19	1	7	896	4/4/4	0.824	12.140	2.798
1e19	2	8	1024	4/4/4	1.033	9.676	2.778
1e19	3	10	1280	4/4/4	1.495	6.688	2.761
1e19	4	11	1408	4/4/4	1.771	5.645	2.759
1e19	5	10	1280	8/4/8	2.856	3.502	2.762
3e19	1	9	1152	4/4/4	1.242	24.150	2.664
3e19	2	10	1280	4/4/4	1.495	20.063	2.644
3e19	3	11	1408	6/3/6	2.553	11.752	2.611
3e19	4	11	1408	8/4/8	3.380	8.874	2.599
3e19	5	12	1536	8/8/8	4.077	7.358	2.604
5e19	1	11	1408	4/4/4	1.771	28.227	2.596
5e19	2	11	1408	6/3/6	2.553	19.587	2.565
5e19	3	11	1408	8/4/8	3.380	14.791	2.540
5e19	4	12	1536	8/4/8	3.888	12.860	2.541
5e19	5	12	1536	8/8/8	4.077	12.264	2.539
5e19	6	14	1792	8/8/8	5.401	9.258	2.537
8e19	1	11	1408	4/8/4	1.934	41.375	2.539
8e19	2	11	1408	8/4/8	3.380	23.665	2.497
8e19	3	12	1536	8/4/8	3.888	20.574	2.490
8e19	4	12	1536	8/8/8	4.077	19.622	2.486
8e19	5	14	1792	8/4/8	5.145	15.550	2.476
8e19	6	16	2048	8/8/8	6.823	11.724	2.478
1e20	1	9	1152	8/8/8	2.485	40.249	2.499
1e20	2	10	1280	8/4/8	2.856	35.015	2.486
1e20	3	13	1664	8/8/8	4.716	21.205	2.452
1e20	4	14	1792	8/8/8	5.401	18.516	2.449
1e20	5	17	2176	8/8/8	7.641	13.087	2.459

ARC-Easy, ARC-Challenge, CommonsenseQA, PIQA, and SocialQA are evaluated using 5-shot multiple-choice prompts and raw choice accuracy. HellaSwag uses a 5-shot ranked-choice formulation with answer likelihood normalized by character length, whereas WinoGrande uses a 5-shot ranked-choice formulation with raw accuracy. MedMCQA and MedQA use 5-shot ranked-choice evaluation with answer likelihood normalized by token length. LAMBADA-Standard and LAMBADA-OpenAI are evaluated zero-shot using greedy next-token completion accuracy in the higher-is-better portion of the table.

Gold-completion and open-ended likelihood evaluation. The likelihood evaluation uses three demonstrations for HumanEval Gold, MBPP Gold, and every MT-MBPP language; zero demonstrations for CoQA and LAMBADA-OpenAI; five demonstrations for DROP, Jeopardy, Natural Questions, and SQuAD; and eight demonstrations for GSM8K Gold. Each task is summarized by the mean negative base-2 log-likelihood of its gold continuations normalized by their UTF-8 byte lengths, measured in bits per byte. The three code likelihood benchmarks use the corrected BPB implementation provided

Table 13 | Prompting and sampling configurations for the generative benchmarks reported in Table 1. The number of samples denotes generated completions per problem.

Benchmark	Shots	Samples	Metric
<i>Mathematical reasoning</i>			
GSM8K	8	1	Pass@1
GSM-Symbolic	8	8	Pass@1
Minerva Math	4	1	Exact match
MATH-500	4	32	Pass@1
<i>Code generation</i>			
BigCodeBench	3	5	Execution pass@1
HumanEval	3	32	Execution pass@1
DS-1000	3	5	Execution pass@1
MBPP	3	32	Execution pass@1
MultiPL-E HumanEval	0	32	Execution pass@1
MultiPL-E MBPP	0	32	Execution pass@1

by the evaluation framework. Lower BPB indicates better performance. These results and their AVG are kept separate from the higher-is-better task scores and Overall AVG.

Generative evaluation. Table 13 lists the prompting and sampling configurations for benchmarks that require free-form generation. The number of samples denotes generated completions per problem and is independent of the inference batch size.

Scoring and aggregation. Mathematical answers are normalized using benchmark-specific answer extractors. GSM8K and GSM-Symbolic use last-number extraction, while Minerva Math and MATH-500 use their corresponding normalized mathematical equivalence rules. The GSM-Symbolic score is macro-averaged equally across its three subsets, and the Minerva Math score is macro-averaged equally across its seven subject categories.

Code completions are executed in isolated task-specific environments using the corresponding official test cases and dependencies. The MultiPL-E HumanEval and MultiPL-E MBPP scores are each macro-averaged equally across the six reported programming languages: C++, C#, Go, Java, JavaScript, and PHP.

The higher-is-better results in Table 1 are reported as percentage points. Within that portion of the table, each section AVG is the unweighted mean of the following constituent results:

- **MMLU AVG:** MMLU-Humanities, MMLU-Social Sciences, MMLU-STEM, MMLU-Other, and MMLU-Pro. The aggregate MMLU row is excluded because it summarizes the same 57 MMLU subjects.
- **MATH AVG:** GSM8K, GSM-Symbolic, Minerva Math, and MATH-500.
- **Code AVG:** BigCodeBench, HumanEval, DS-1000, MBPP, MultiPL-E HumanEval, and MultiPL-E MBPP.
- **MC-STEM AVG:** ARC-Easy and ARC-Challenge.
- **MC-Non-STEM AVG:** PIQA, CommonsenseQA, and SocialQA.
- **GenQA AVG:** HellaSwag, WinoGrande, LAMBADA-Standard, LAMBADA-OpenAI, MedMCQA, and MedQA.

The higher-is-better Overall AVG is the unweighted mean of the 26 constituent benchmark results listed above, rather than the mean of the section AVGs. It excludes the aggregate MMLU row, which

is shown only as a summary of the 57 MMLU subjects, and excludes all intermediate AVG rows.

For MT-MBPP Gold, BPB is first macro-averaged equally across its 17 language subsets. The lower-is-better AVG is then the unweighted mean of the ten benchmark-level BPB results listed in the likelihood portion of the table. Consequently, MT-MBPP Gold receives the same weight as each of the other nine benchmarks, rather than 17 times their weight. The BPB AVG is computed independently and is excluded from every domain AVG and from the higher-is-better Overall AVG.

Every displayed delta is computed as the NCP score minus the corresponding Vanilla score. Therefore, a positive delta denotes an improvement for the higher-is-better task scores, whereas a negative delta denotes an improvement for the lower-is-better BPB scores.

Domain-adaptation evaluation. For the VQ training study in Section 5.1, the code-adapted model is evaluated on HumanEval [Chen et al., 2021] and MBPP [Austin et al., 2021] using the augmented HumanEval+ and MBPP+ test suites provided by EvalPlus [Liu et al., 2023]. The mathematics-adapted model is evaluated on GSM8K [Cobbe et al., 2021] and MATH-500 [Hendrycks et al., 2021b, Lightman et al., 2023]. The knowledge-adapted model is evaluated on TriviaQA [Joshi et al., 2017] using its standard answer-matching protocol. General-capability retention is measured using MMLU [Hendrycks et al., 2021a], ARC-Easy [Clark et al., 2018], HellaSwag [Zellers et al., 2019], and Natural Questions [Kwiatkowski et al., 2019]. These auxiliary domain-adaptation results are not included in Table 1 or either of its aggregate scores.

Detailed results on the stage-1 checkpoint trajectory: Table 14 shows detailed results on the stage-1 checkpoint trajectory.

Table 14 | NCP-ArchPreview stage-1 downstream performance across training checkpoints. The reported benchmarks follow the same grouping as Table 1. The likelihood block reports BPB (lower is better) and is aggregated separately from the higher-is-better Overall AVG.

Dataset	100K	200K	300K	400K	500K	600K	700K	800K	900K	1.0M	1.1M	1.2M	1.3M	Final
MMLU														
MMLU-Humanities	54.38	58.12	62.77	63.02	63.57	65.10	64.57	65.14	65.69	66.16	66.64	67.74	68.26	68.28
MMLU-Social Sciences	60.38	64.61	66.97	67.09	69.65	69.92	70.54	71.57	72.05	71.73	72.44	73.62	73.41	73.97
MMLU-STEM	46.06	49.17	50.32	50.89	53.96	51.47	53.62	53.86	54.48	54.12	54.43	54.05	55.45	55.53
MMLU-Other	54.65	58.13	61.53	61.78	63.64	62.89	64.23	65.42	65.07	65.46	66.04	66.93	66.10	65.63
MMLU	53.08	56.66	59.42	59.74	61.83	61.27	62.29	63.00	63.34	63.36	63.86	64.46	64.77	64.80
MMLU-Pro	15.59	16.89	17.15	18.46	18.65	18.73	19.27	19.30	19.66	20.45	20.15	20.17	20.25	20.26
MATH														
GSM8K	23.43	31.16	32.75	34.87	39.04	40.49	40.26	41.77	42.15	43.37	42.38	44.58	45.34	45.26
GSM-Symbolic	8.55	13.14	14.37	15.88	17.12	19.13	19.32	20.12	20.81	21.86	20.37	22.05	23.05	22.80
Minerva	7.49	8.71	10.45	10.51	11.70	12.26	11.43	12.46	13.76	14.27	14.46	14.83	14.62	15.63
MATH-500	6.98	7.93	9.77	9.90	10.42	10.36	11.16	11.99	12.67	12.65	13.81	13.73	13.66	14.48
Code														
BigCodeBench	10.56	14.70	17.61	16.26	19.04	19.51	19.14	20.46	21.42	21.75	20.93	21.93	22.54	22.79
HumanEval	20.05	23.30	27.02	25.06	25.93	26.14	28.87	26.33	29.55	30.30	30.53	30.30	29.40	31.38
DS-1000	8.84	12.74	14.44	13.60	16.12	16.96	16.90	17.00	18.70	19.70	18.80	19.86	20.42	20.40
MBPP	27.71	27.46	31.15	31.86	30.43	31.66	33.26	34.59	35.29	35.04	34.99	36.64	36.09	35.91
MultiPL-E HumanEval	15.17	16.81	18.26	18.44	18.16	19.18	20.33	22.38	22.17	21.18	21.26	22.07	21.36	22.27
MultiPL-E MBPP	27.64	27.15	30.34	29.54	29.86	32.20	32.04	34.65	34.00	34.18	33.36	34.64	34.86	33.96
MC-STEM														
ARC-E	82.24	86.24	88.38	90.19	89.10	90.40	91.84	91.84	92.05	91.88	92.05	92.26	92.97	92.30
ARC-C	65.61	69.97	72.78	74.74	75.43	76.11	77.30	78.16	79.27	78.67	78.92	81.31	80.46	81.57
MC-Non-STEM														
PiQA	67.08	72.20	73.61	75.95	76.12	77.58	77.58	79.54	80.20	79.98	79.92	80.85	80.96	80.85
CommonsenseQA	62.82	68.80	70.76	70.52	71.99	71.42	72.73	73.55	73.71	73.55	73.14	73.14	74.69	74.04
SocialQA	59.42	63.56	65.30	66.79	67.30	67.91	69.09	69.70	68.12	68.78	69.09	69.50	69.45	69.24
GenQA														
HellaSwag	61.60	62.70	63.60	64.75	63.55	64.10	65.20	66.05	65.35	66.05	66.05	65.70	66.00	66.05
WinoGrande	50.59	49.72	51.14	49.25	50.59	48.54	49.88	49.57	49.41	50.28	48.78	49.25	50.51	51.22
LAMBADA-Standard	62.45	63.34	64.60	65.90	66.00	66.02	67.86	66.89	65.90	65.03	67.01	65.85	66.54	66.89
LAMBADA-OpenAI	67.46	68.83	68.29	70.81	70.83	70.79	71.61	71.67	70.42	71.47	71.80	72.00	71.51	72.21
MedMCQA	30.98	31.81	31.80	31.64	32.08	32.14	32.32	32.69	32.88	32.91	32.69	33.15	33.16	33.21
MedQA	32.99	34.72	35.74	36.53	36.92	36.53	37.55	38.88	39.28	38.57	37.94	38.49	39.67	38.96
AVG	39.64	42.38	44.27	44.78	45.66	46.06	46.84	47.52	47.85	48.05	48.00	48.64	48.87	49.04
Likelihood (BPB; ↓)														
HumanEval Gold	0.374	0.382	0.376	0.358	0.370	0.381	0.375	0.361	0.366	0.376	0.364	0.359	0.363	0.364
MBPP Gold	0.515	0.513	0.505	0.500	0.495	0.493	0.495	0.487	0.489	0.485	0.473	0.473	0.479	0.473
MT-MBPP Gold	0.472	0.461	0.445	0.446	0.448	0.447	0.457	0.453	0.449	0.441	0.429	0.440	0.434	0.434
CoQA	0.403	0.404	0.377	0.384	0.382	0.377	0.374	0.373	0.379	0.366	0.372	0.357	0.367	0.370
DROP	4.922	4.831	4.985	4.512	4.592	4.479	4.424	4.365	4.464	4.444	4.424	4.304	4.252	4.397
GSM8K Gold	0.431	0.419	0.418	0.402	0.406	0.405	0.406	0.402	0.404	0.403	0.398	0.394	0.404	0.400
Jeopardy	0.429	0.395	0.383	0.391	0.393	0.388	0.374	0.372	0.380	0.365	0.360	0.355	0.350	0.347
LAMBADA-OpenAI	0.332	0.323	1.061	0.304	0.301	0.300	0.295	0.293	0.297	0.296	0.291	0.289	0.291	0.288
Natural Questions	0.993	0.969	0.957	0.939	0.942	0.932	0.917	0.930	0.923	0.914	0.915	0.892	0.895	0.884
SQuAD	0.162	0.159	0.158	0.154	0.154	0.135	0.146	0.143	0.138	0.148	0.151	0.149	0.145	0.151
AVG (↓)	0.903	0.886	0.967	0.839	0.848	0.834	0.826	0.818	0.829	0.824	0.818	0.801	0.798	0.811